프로그래밍

차례

항목을 누르면 해당 장으로 이동합니다. PDF 책갈피에서도 과목을 찾을 수 있습니다.

프로그래밍 · 30분 학습 방법 6

프로그래밍 001회차 | 02-01 원리 I 7

프로그래밍 002회차 | 02-01 원리II 8

프로그래밍 003회차 | 02-01 예제·적용 9

프로그래밍 004회차 | 02-01 직접 문제 10

프로그래밍 005회차 | 02-01 해설·재현 12

프로그래밍 006회차 | 02-01 원리 I 13

프로그래밍 007회차 | 02-01 원리II 14

프로그래밍 008회차 | 02-01 예제·적용 15

프로그래밍 009회차 | 02-01 직접 문제 16

프로그래밍 010회차 | 02-01 해설·재현 18

프로그래밍 011회차 | 02-02 원리 I 20

프로그래밍 012회차 | 02-02 원리II 21

프로그래밍 013회차 | 02-02 예제·적용 23

프로그래밍 014회차 | 02-02 직접 문제 24

프로그래밍 015회차 | 02-02 해설·재현 26

프로그래밍 016회차 | 02-02 원리 I 27

프로그래밍 017회차 | 02-02 원리II 28

프로그래밍 018회차 | 02-02 예제·적용 29

프로그래밍 019회차 | 02-02 직접 문제 30

프로그래밍 020회차 | 02-02 해설·재현 32

프로그래밍 021회차 | 02-03 원리 I 34

프로그래밍 022회차 | 02-03 원리II 35

프로그래밍 023회차 | 02-03 예제·적용 37

프로그래밍 024회차 | 02-03 직접 문제 38

프로그래밍 025회차 | 02-03 해설·재현 40

프로그래밍 026회차 | 02-03 원리 I 41

프로그래밍 027회차 | 02-03 원리II 42

프로그래밍 028회차 | 02-03 예제·적용 43

프로그래밍 029회차 | 02-03 직접 문제 44

프로그래밍 030회차 | 02-03 해설·재현 46

프로그래밍 031회차 | 02-04 원리 I 48

프로그래밍 032회차 | 02-04 원리II 50

프로그래밍 033회차 | 02-04 예제·적용 51

프로그래밍 034회차 | 02-04 직접 문제 52

프로그래밍 035회차 | 02-04 해설·재현 54

프로그래밍 036회차 | 02-04 원리 I 55

프로그래밍 037회차 | 02-04 원리II 56

프로그래밍 038회차 | 02-04 예제·적용 57

프로그래밍 039회차 | 02-04 직접 문제 58

프로그래밍 040회차 | 02-04 해설·재현 60

프로그래밍 041회차 | 02-05 원리 I 62

프로그래밍 042회차 | 02-05 원리II 63

프로그래밍 043회차 | 02-05 예제·적용 64

프로그래밍 044회차 | 02-05 직접 문제 65

프로그래밍 045회차 | 02-05 해설·재현 67

프로그래밍 046회차 | 02-05 원리 I 68

프로그래밍 047회차 | 02-05 원리II 69

프로그래밍 048회차 | 02-05 예제·적용 70

프로그래밍 049회차 | 02-05 직접 문제 71

프로그래밍 050회차 | 02-05 해설·재현 73

프로그래밍 051회차 | 02-06 원리 I 75

프로그래밍 052회차 | 02-06 원리II 76

프로그래밍 053회차 | 02-06 예제·적용 77

프로그래밍 054회차 | 02-06 직접 문제 78

프로그래밍 055회차 | 02-06 해설·재현 80

프로그래밍 056회차 | 02-06 원리 I 81

프로그래밍 057회차 | 02-06 원리II 82

프로그래밍 058회차 | 02-06 예제·적용 83

프로그래밍 059회차 | 02-06 직접 문제 84

프로그래밍 060회차 | 02-06 해설·재현 86

프로그래밍 061회차 | 02-07 원리 I 88

프로그래밍 062회차 | 02-07 원리II 90

프로그래밍 063회차 | 02-07 예제·적용 91

프로그래밍 064회차 | 02-07 직접 문제 92

프로그래밍 065회차 | 02-07 해설·재현 94

프로그래밍 066회차 | 02-07 원리 I 95

프로그래밍 067회차 | 02-07 원리II 96

프로그래밍 068회차 | 02-07 예제·적용 97

프로그래밍 069회차 | 02-07 직접 문제 98

프로그래밍 070회차 | 02-07 해설·재현 100

프로그래밍 071회차 | 02-08 원리 I 102

프로그래밍 072회차 | 02-08 원리II 103

프로그래밍 073회차 | 02-08 예제·적용 105

프로그래밍 074회차 | 02-08 직접 문제 106

프로그래밍 075회차 | 02-08 해설·재현 108

프로그래밍 076회차 | 02-08 원리 I 109

프로그래밍 077회차 | 02-08 원리II 110

프로그래밍 078회차 | 02-08 예제·적용 111

프로그래밍 079회차 | 02-08 직접 문제 112

프로그래밍 080회차 | 02-08 해설·재현 114

근거·범위·검증 상태 116

30분 학습 방법

30분 안에서 진행하기

복습 3분 → 학습 25분 → 기록 2분으로 진행합니다. 시간이 끝나면 멈추고, 남은 범위는 다음 학습일에 같은 회차에서 이어갑니다. 한 회차를 하루에 반드시 끝내야 하는 것은 아닙니다.

원리 → 문제 → 분리 해설

각 5회차 묶음에서 앞 3회차는 원리와 예제를 읽고, 4회차는 직접 문제를 풀며, 5회차는 해설을 비교하고 답 없이 다시 풉니다. 완료 기준은 읽은 쪽수보다 용어·조건을 설명하고 풀이를 재현할 수 있는지입니다.

강조와 보완 자료 활용

노란색으로 표시한 핵심 조건을 먼저 확인한 뒤, 예제에서 그 조건이 어떻게 쓰였는지 설명합니다. 책 뒤 117쪽부터는 반례·손풀이·추가 확인 문제가 있습니다. 해당 주제가 어려운 날의 학습 시간 안에 골라 읽습니다. 추가 문제의 해설은 별도 절에 있습니다.

공부 기록은 한 곳에

Obsidian의 「공부기록」에 과목·회차·실제 공부 시간·다음 시작점을 남깁니다. 「하루공부」 템플릿을 같은 기록에 추가해서 사용합니다. PDF의 강조나 필기는 진도 표를 자동으로 변경하지 않습니다.

프로그래밍 001회차 | 02-01 원리 I

언어 실행·자료형·정수 연산·비트 연산 · 과목 1/80회차 · 기록 묶음 01의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

C# 개발 경력을 공부로 연결하기

이미 알고 있는 변수·조건문을 다시 외우는 대신, 같은 코드처럼 보여도 언어 규칙이 달라지는 지점을 확인한다. 전산필기에서는 C·Java 등의 출력 해석이 나올 수 있으므로 코딩테스트 주력 언어를 C++·Python으로 정해도 프로그래밍 이론을 두 언어로만 한정하지 않는다. 여기의 C++ 예제는 C++17, Python 예제는 Python3의 기초 기능을 기준으로 한다. 실제 시험의 지원 버전은 해당 공고·시험 안내에서 확인한다.

1. 컴파일과 실행

컴파일은 소스의 구조와 의미를 분석해 다른 표현으로 변환하는 과정이다. 전처리·어휘 분석·구문 분석·의미 분석·중간 표현·최적화·코드 생성·링크라는 전형적인 설명을 학습하되 모든 구현이 똑같은 별도 단계로 실행된다고 단정하지 않는다. 링크는 여러 목적 파일과 라이브러리의 기호를 연결한다. 문법이 틀리면 구문 단계의 오류, 없는 함수 정의를 사용하면 링크 오류가 될 수 있다.

C#과 Java는 중간 코드를 런타임이 실행하거나 JIT 컴파일할 수 있다. Python도 CPython에서는 바이트코드로 컴파일해 실행한다. "컴파일 언어는 실행 전에 모두 기계어, 인터프리터 언어는 한 줄씩 읽기만"이라는 이분법은 구현 현실을 지나치게 단순화한다. 정적·동적 타입은 타입 검사 시점의 특성이며 컴파일·인터프리트와 동일한 분류가 아니다.

2. 타입·범위·변환

정수·실수·문자·불리언·컨테이너의 역할을 분리한다. C++ int의 정확한 크기는 구현에 따라 달라질 수 있으므로 시험이 32비트를 가정했다면 그 가정을 적는다. Python int는 고정32비트 범위로 제한되지 않지만 메모리·연산 비용이 무한히 공짜인 것은 아니다. C# int와 Java int는32비트 부호 있는 정수다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 002회차 | 02-01 원리ㅍ

언어 실행·자료형·정수 연산·비트 연산 · 과목 2/80회차 · 기록 묶음 01의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

손풀이 예제 A: 32비트 부호 있는 정수의 수학적 표현 범위는 -2^(31)..2^(31)-1, 즉 -2,147,483,648..2,147,483,647이다. 1,000,000,000×3의 정확한 값3,000,000,000은 이 범위를 넘는다. C++에서는 긴 정수 변수에 결과를 대입하는 것만으로 중간 int 곱셈 overflow를 방지하지 못한다. 1LL*a*b처럼 계산 이전에 충분한 타입으로 올리는 방법을 고려한다.

C++ 부호 있는 정수 overflow는 일반적으로 정의되지 않은 동작(UB)이므로 "무조건 음수로 돌아온다"고 답하지 않는다. 부호 없는 N비트 정수 연산은 2^N을 법으로 한 연산으로 설명한다. C#은 checked/unchecked 문맥을 읽어야 하고 Java int의 overflow는 정해진 wrap 규칙이 있다. 서로의 규칙을 옮겨 답하지 않는다.

3. 나눗셈과 나머지

C++17에서 정수 나눗셈은0 쪽으로 소수 부분을 버린다. Python의 //는 아래 방향으로 내림한다. 양수에서는 결과가 같지만 음수에서는 차이가 드러난다.

식	C++17 정수식	Python3
7 / 3 또는 7 // 3	2	2
-7 / 3 또는 -7 // 3	-2	-3
-7 % 3	-1	2

손풀이 예제 B: Python에서 -7=(-3)×3+2이므로 나머지2다. C++에서 -7=(-2)×3+(-1)이다. a=q×b+r의 관계는 지키면서 몫의 버림 방향이 다르다. Python의 /는 정수 입력이어도 실수 나눗셈이다. 7/3은 약2.333..., 7//3은2다. 음수 모듈러가 필요한 문제에서는 언어 규칙에 맞게 정규화하고 나눗셈0을 막는다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 003회차 | 02-01 예제·적용

언어 실행·자료형·정수 연산·비트 연산 · 과목 3/80회차 · 기록 묶음 01의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

4. 실수와 비트

이진 부동소수점으로0.1 같은 값이 정확히 표현되지 않을 수 있다. 금융 금액을 무작정 double로 계산하는 것과 정수 단위·십진 자료형으로 계산하는 것의 차이를 이해한다. 작은 오차 허용 여부는 문제의 정답 판정 규칙을 따른다. "모든 실수 비교는 절대로 ==를 쓰면 안 된다"라는 극단적 문장보다 표현과 요구를 확인한다.

AND(&), OR(|), XOR(^), NOT(~), shift는 비트를 대상으로 한다. 논리 연산 &&/|| 또는 and/or와 구별한다. 양의 값 5=0101,3=0011을4비트 그림으로 쓰면 AND=0001(1), OR=0111(7), XOR=0110(6)이다. 1<<3은 해당 표현이 유효한 타입·이동 범위에서8이다. 음수 shift나 범위를 넘는 이동 수의 규칙은 언어·버전에 따라 달라질 수 있어 본 교재의 출력 예제에서 피한다.

직접 연습·완료 기준

- 32비트 int의 최댓값을 적고 10만개의10만 합10^(10)이 넘는지 확인한다.
- Python -10//3=-4, -10%3=2를 관계식으로 검산한다.
- 6(0110) XOR 3(0011)=5(0101)을 직접 계산한다.

세 언어의 overflow를 한 규칙으로 묶지 않고 나눗셈·타입 변환 시점을 표시할 수 있으면 완료한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 004회차 | 02-01 직접 문제

언어 실행·자료형·정수 연산·비트 연산 · 과목 4/80회차 · 기록 묶음 01의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q01

C++17 정수 -7/3과 Python3 -7//3의 결과를 순서대로 고르면?

1. -2,-3

2. -3,-3

3. -2,-2

4. -3,-2

내 선택·근거: _____

B02Q02

C++에서 부호 있는 int 곱셈이 표현 범위를 넘는다. 일반적인 표준 관점의 올바른 설명은?

1. 항상0이다.

2. 일반적으로 정의되지 않은 동작이므로 특정 결과를 보장할 수 없다.

3. 항상 최솟값으로 포화한다.

4. 반드시 Python처럼 큰 정수로 자동 확장한다.

내 선택·근거: _____

B02Q03

5와3을 각각0101,0011로 표현한다. XOR 결과는?

1. 6

2. 1

3. 7

4. 8

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 005회차 | 02-01 해설·재현

언어 실행·자료형·정수 연산·비트 연산 · 과목 5/80회차 · 기록 묶음 01의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q01

정답: 1번

C++17은0 방향 버림으로-2, Python //는 아래 방향 내림으로-3이다. ②은 C++에 floor를 적용했고, ③은 Python에0 방향 버림을 적용했고, ④는 두 규칙을 뒤집었다. 나머지도 C++ -1, Python2로 관계식을 검산한다.

B02Q02

정답: 2번

부호 있는 overflow를 wrap 결과로 단정하지 않는다. ①·③는 표준 보장 없는 수치를 정했고, ④는 Python 정수 모델을 C++에 옮겼다. 충분한 타입으로 계산 전에 올려야 하는 예와 부호 없는 모듈러 규칙을 함께 복습한다.

B02Q03

정답: 1번

다른 비트만1이므로0110=6이다. ②은 AND, ③은 OR 결과이고, ④는 일반 덧셈 결과다. 비트 XOR와 산술 덧셈은 다르다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 006회차 | 02-01 원리 I

언어 실행·자료형·정수 연산·비트 연산 · 과목 6/80회차 · 기록 묶음 02의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 실습: 계산 타입은 대입 전에 결정된다

int a=1000000000,b=3을 충분히 큰 결과 변수에 넣는다고 하자. C++에서 long long result=a*b는 먼저 int끼리 곱하려 하므로 그 타입의 범위를 넘는 중간 계산이 문제가 된다. long long result=1LL*a*b는 첫 곱부터 long long으로 계산한다. 이 예제에서 정확한 값은3000000000이다.

상수 식의 overflow,실행 중 overflow,형 변환의 규칙은 언어마다 다르므로 결과 숫자만 외우지 않는다. C#에서도 checked 문맥의 실행 중 overflow와 unchecked 문맥을 구별하고,컴파일 시 상수 식의 진단은 별도로 확인한다. Python에서는 같은 곱이 정수 값으로 계산되지만 매우 큰 정수를 계속 키우면 비트 길이에 따른 메모리·연산 비용이 생긴다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 007회차 | 02-01 원리ㅍ

언어 실행·자료형·정수 연산·비트 연산 · 과목 7/80회차 · 기록 묶음 02의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

추가 비트 연습: 상태의0번·2번 비트를 켜려면0b0101을 만들 수 있다. 상태5와 mask2의 AND는0이므로1번 비트는 꺼져 있다. "비트 번호"는0 기반이라는 계약을 적는다. NOT의 결과를4비트 그림으로 보여 주는 것과 Python의 무한 부호 확장 정수에서 ~5=-6이라는 동작을 혼동하지 않는다.

강화 응용: 조건을 바꾸어 다시 증명하기

언어 간 정수 계약을 먼저 정한다. C++17의 부호 있는 정수 덧셈에서 표현 범위를 넘는 오버플로는 정의되지 않은 동작이며 '자동으로 한 바퀴 돈다'고 답하지 않는다. 부호 없는 정수는 해당 형의 범위에 맞춘 모듈러 연산이다. Python3 int는 임의 정밀도지만 비트 수가 커지면 연산 비용과 메모리 사용도 증가한다. 정답의 최종 합만 범위에 들어간다고 중간 합이 안전한 것은 아니다. long long의 최댓값 M에 1을 더하고 나중에 1을 빼면 최종 수학적 값 M과 관계없이 첫 덧셈이 위험하다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 008회차 | 02-01 예제·적용

언어 실행·자료형·정수 연산·비트 연산 · 과목 8/80회차 · 기록 묶음 02의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

Python3에서 -5//2=-3, -5%2=1이며 (-3)*2+1=-5다. C++17에서 -5/2=-2,-5%2=-1이며 (-2)*2-1=-5다. 두 언어 모두 몫·나머지 관계식은 유지되지만 음수 나눗셈의 몫 버림 규칙이 다르다. 배열 인덱스의 주기 순환에는 언어 차이를 확인하고 음수 나머지를 비음수로 정규화한다.

C++에서 부호 있는 인덱스 -1과 컨테이너의 부호 없는 size()를 비교하면 의도와 다를 수 있다. 64비트 size_t로 변환되는 비교에서는 -1이 큰 부호 없는 값이 되어 -1<3이 false다. '음수니까 당연히 작다'는 수학 판단을 그대로 옮기지 않는다. 인덱스 유효 조건은 i≥0을 먼저 확인한 다음 안전한 형 변환과 i<size를 확인한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 009회차 | 02-01 직접 문제

언어 실행·자료형·정수 연산·비트 연산 · 과목 9/80회차 · 기록 묶음 02의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q25

Python3에서 -5//2, -5%2의 값을 순서대로 고르면?

1. (-2,-1)

2. (-3,-1)

3. (-2,1)

4. (-3,1)

내 선택·근거: _____

B02Q26

C++17에서 int가 32비트 부호 있는 형이다. INT_MAX+1을 int 연산으로 계산할 때 표준에 맞는 설명은?

1. 반드시 INT_MIN이 된다.

2. 반드시 0이 된다.

3. 표준이 자동으로 long long으로 바꾸어 정확히 계산한다.

4. 정의되지 않은 동작이며 정답을 순환 값으로 고정할 수 없다.

내 선택·근거: _____

B02Q27

C++17, int 32비트·size_t 64비트 부호 없는 형을 가정한다. int i=-1; size_t n=3;에서 i<n의 결과와 이유는?

1. true, 음수는 모든 자연수보다 작다.

2. 컴파일 오류, 부호가 다르면 비교할 수 없다.

3. true, n이 항상 int로 변환된다.

4. false, i가 size_t의 큰 값으로 변환되어 비교된다.

내 선택·근거: _____

W02Q01

C++17과 Python3에서 -5를 2로 나눈 몫·나머지를 비교하라. 또한 C++17 부호 있는 오버플로와 Python 큰 정수에 대한 안전/비용 차이를 설명하라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 010회차 | 02-01 해설·재현

언어 실행·자료형·정수 연산·비트 연산 · 과목 10/80회차 · 기록 묶음 02의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q25

정답: 4번

풀이와 보기별 판정

「(-3,1)」 - 정답. 내림 몫 -3과 나머지 1로 -5=(-3)*2+1이 성립한다.

「(-2,-1)」 - 오답. C++17의 0 방향 버림 규칙 결과다.

「(-3,-1)」 - 오답. 몫 -3이면 나머지는 -5-(-6)=1이어야 한다.

「(-2,1)」 - 오답. 몫과 나머지가 원래 수를 재구성하지 못한다.

B02Q26

정답: 4번

풀이와 보기별 판정

「반드시 INT_MIN이 된다.」 - 오답. 부호 있는 오버플로에 부호 없는 모듈러 규칙을 적용했다.

「정의되지 않은 동작이며 정답을 순환 값으로 고정할 수 없다.」 - 정답. 표현 범위를 넘는 부호 있는 덧셈은 정의되지 않은 동작이다.

「반드시 0이 된다.」 - 오답. 0이라는 표준 보장은 없다.

「표준이 자동으로 long long으로 바꾸어 정확히 계산한다.」 - 오답. 두 피연산자가 int이면 long long으로 자동 확대되지 않는다.

B02Q27

정답: 4번

풀이와 보기별 판정

「true, 음수는 모든 자연수보다 작다.」 - 오답. 통상 산술 변환을 적용하지 않은 수학적 판단이다.

「컴파일 오류, 부호가 다르면 비교할 수 없다.」 - 오답. 부호가 달라도 비교는 가능하지만 변환을 주의해야 한다.

「false, i가 size_t의 큰 값으로 변환되어 비교된다.」 - 정답. 64비트 부호 없는 형이 32비트 int의 모든 비음수 값 범위를 포함해 i를 unsigned로 변환한다.

「true, n이 항상 int로 변환된다.」 - 오답. 주어진 형 범위에서는 n이 int로 변환되지 않는다.

W02Q01

C++17과 Python3에서 -5를 2로 나눈 몫·나머지를 비교하라. 또한 C++17 부호 있는 오버플로와 Python 큰 정수에 대한 안전/비용 차이를 설명하라.

모범답안

C++17은 -5/2=-2,-5%2=-1이고 Python3은 -5//2=-3,-5%2=1이다. 각각 q*2+r=-5를 만족한다. C++17 정수 나눗셈은 0 방향 버림, Python //는 음의 무한대 방향 내림이다. C++ 부호 있는 오버플로는 정의되지 않은 동작이므로 모든 중간 연산이 범위 안이어야 한다. Python int는 더 큰 값을 정확히 표현하지만 비트 길이가 커지면 연산·메모리 비용도 커져 모든 연산을 고정 O(1)로 보지 않는다.

10점 자가 채점 기준

- 두 언어의 몫·나머지 4개 값: 4점
- 관계식과 버림 방향: 2점
- C++ 중간 범위/정의되지 않은 동작: 2점
- Python 임의 정밀도와 비트 길이 비용: 2점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 011회차 | 02-02 원리 I

조건·반복·입출력·코드 추적 · 과목 11/80회차 · 기록 묶음 03의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

코드 해석은 표를 만드는 작업이다

프로그램을 읽으면서 머릿속으로 계속 수정하지 않는다. 반복 번호, 조건 평가, 바뀐 변수, 출력 시점의 열을 만들어 추적한다. 코드가 요구하는 출력과 최종 변수 값이 다른 경우를 구별하고, 증감 연산의 결과가 모호한 식은 언어 규칙을 먼저 확인한다.

1. 조건과 단락 평가

C++의 &&, ||는 왼쪽을 먼저 평가하고 결과가 정해지면 오른쪽을 생략한다. Python의 and, or도 단락 평가를 하지만 반드시 bool만 반환하는 것은 아니다. 0 or 7은7, [] and 5는[]를 반환한다. Python에서 빈 컨테이너·0·None은 거짓으로 해석되며 내용이 있는 목록은 참이다.

손풀이 예제 A: x=0이고 x != 0 and 10/x > 2라면 왼쪽이 거짓이어서 오른쪽 나눗셈을 수행하지 않는다. 안전한 순서는 "유효성 검사 먼저→사용"이다. 같은 식에서 and를 비트 연산 &로 기계적으로 바꾸면 단락 평가의 보호를 기대할 수 없다. C++에서는 배열 인덱스 사용 전에 범위 조건을 확인한다.

Python의 1 < x < 5는 연쇄 비교이고 x를 한 번 평가하는 언어 의미를 갖는다. C++의 1 < x < 5는 같은 수학적 의미가 아니다. C++은 1 < x && x < 5처럼 쓴다. 언어의 문법이 비슷하다는 이유로 식 전체를 그대로 옮기지 않는다.

2. for·while의 실행 수

Python range(n)은0..n-1이고 range(a,b)는 b를 포함하지 않는다. C++ for의 초기화는 한 번, 조건은 각 반복 전, 증감은 본문 뒤에 실행된다. break는 가장 안쪽 반복을 끝내고 continue는 해당 반복의 나머지 본문을 건너뛴다. 중첩 루프의 바깥까지 모두 종료한다는 뜻이 아니다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 012회차 | 02-02 원리ㅍ

조건·반복·입출력·코드 추적 · 과목 12/80회차 · 기록 묶음 03의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 B:

```
s = 0
for i in range(1, 6):
    if i % 2 == 0:
        continue
    s += i
print(s)
```

i는1,2,3,4,5다. 짝수2·4를 건너뛰고1+3+5=9를 출력한다. continue가 반복 자체를 끝내는 것은 아니다. i==4에서 break하도록 바꾸면4에서 종료해1+2+3=6이 된다. 두 코드를 섞어 외우지 않는다.

while에서는 진행이 보장되는지 본다. 포인터가 특정 분기에서 증가하지 않으면 무한 루프가 생길 수 있다. while lo < hi의 이분 탐색에서 lo=mid로만 바꾸면 lo+1==hi인 상황에서 mid==lo가 되어 진행되지 않을 수 있다. 상태가 줄어드는 양 또는 증가하는 양을 설명해야 한다.

3. 입력·출력과 빈 입력

코딩테스트는 프롬프트나 디버그 출력 없이 요구 형식만 출력한다. 입력이 한 줄에만 있다는 보장이 없을 때 whitespace 토큰으로 읽는 방식이 편리하지만 전체 데이터를 메모리에 올리는 비용도 본다. 문자열에 공백이 의미가 있으면 split으로 분리하면 정보가 사라지므로 줄 단위 입력을 사용한다.

```
import sys
def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    values = data[1:]
    if len(values) != n:
        raise ValueError("expected n values")
    print(sum(values))
if __name__ == "__main__":
    main()
```

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 013회차 | 02-02 예제·적용

조건·반복·입출력·코드 추적 · 과목 13/80회차 · 기록 묶음 03의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

본 예제는 입력이 "n 다음에 정수 n개"라는 학습 계약이고 잘못된 입력은 예외로 드러낸다. 공식 시험이 올바른 입력을 보장하면 오류 검증 방식은 더 간단할 수 있다. n=0이면 합0을 출력한다. 입력이 아예 비어 있으면 출력 없이 끝낸다. 둘은 다른 입력이다.

C++에서는 ios::sync_with_stdio(false); cin.tie(nullptr);를 성능 선택으로 사용할 수 있다. 이때 C stdio와의 섞어 쓰기를 무심코 적용하지 않는다. 정수 합은 범위에 맞는 long long으로 누적하고, 줄바꿈 문자와 형식을 맞춘다. 성능 옵션 자체가 정답을 보장하지는 않는다.

4. 연산 우선순위와 위험한 식

우선순위는 식이 묶이는 순서, 평가 순서는 피연산자를 실제로 계산하는 순서다. 둘은 같지 않다. C/C++의 함수 인자 평가나 한 식 안의 여러 증감은 언어 버전과 sequencing 규칙을 읽어야 한다. i++ + ++i 같은 식에 임의의 왼쪽 우선 규칙을 붙여 단정하지 않는다. 시험에서도 정의되지 않은 동작인지 먼저 판별해야 할 수 있다.

예제 C: C++17의 안전한 분리 코드 int x=2; int a=x++; int b=++x;에서 첫 문장 뒤 a=2,x=3, 다음 뒤 b=4,x=4다. 각 전체 표현식의 종료를 기준으로 바뀐 상태를 적으면 순서를 안전하게 설명할 수 있다. Python에는 x++ 증감 연산자가 없으므로 x+=1 등으로 표현한다.

직접 연습과 완료 기준

1. range(2,8,2)는2,4,6이며8은 제외된다.

2. sum=0, i=1부터5까지 i==3에서 break하면 sum=1+2=3이다.

3. Python "A" or "B"는 "A"다.

조건이 오른쪽까지 실행되는지, 반복 경계가 포함인지, 출력 시점이 어디인지 표시할 수 있으면 완료한다. 코드는 줄을 읽는 속도보다 상태를 빠뜨리지 않는 정확성이 먼저다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 014회차 | 02-02 직접 문제

조건·반복·입출력·코드 추적 · 과목 14/80회차 · 기록 묶음 03의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q04

Python3에서 0 or 7의 값은?

1. False

2. True

3. 0

4. 7

내 선택·근거: _____

B02Q05

Python range(1,6)에서 짝수 i는 continue하고 나머지 i를 합한다. 합은?

1. 6

2. 9

3. 10

4. 15

내 선택·근거: _____

B02Q06

C++17 코드 int x=2; int a=x++; int b=++x; 후 a,b,x는?

1. 3,4,4

2. 2,3,3

3. 2,4,4

4. 3,3,4

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 015회차 | 02-02 해설·재현

조건·반복·입출력·코드 추적 · 과목 15/80회차 · 기록 묶음 03의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q04

정답: 4번

or는 왼쪽이 거짓이면 오른쪽 값을 반환한다. 항상 bool을 반환하지 않는다. ①·②는 논리 평가와 반환 객체를 혼동하고, ③은 단락 평가에서 거짓 왼쪽을 반환한다고 잘못 읽었다.

B02Q05

정답: 2번

범위는1,2,3,4,5이며 홀수1+3+5=9다. ①은 break 상황이나 일부만 합한 값, ③은 잘못된 경계·조건, ④는 continue를 무시한 전체 합이다.

B02Q06

정답: 3번

분리된 전체 표현식에서 후위 x++는 기존2를 a에 넣고 x=3, 전위 ++x는4로 올린 값을 b에 넣는다. ①는 후위를 전위로, ②은 두 번째 증가를 누락했고, ④는 두 증감의 반환값을 섞었다. 한 식 안의 모호한 다중 증감으로 일반화하지 않는다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 016회차 | 02-02 원리 I

조건·반복·입출력·코드 추적 · 과목 16/80회차 · 기록 묶음 04의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 손풀이: 중첩 반복의 continue

다음 코드는 별표가 아니라 정수 합을 계산한다.

```
s = 0
for i in range(3):
    for j in range(3):
        if i == j:
            continue
        s += i + j
print(s)
```

대각선(0,0),(1,1),(2,2)만 생략한다. 나머지 여섯 쌍의 합은1,2,1,3,2,3으로 총12다. continue는 안쪽 반복만 이어가며 바깥 i를 건너뛰지 않는다. 전체 쌍9개에서 대각선 합0+2+4를 제외하는 검산도 가능하다.

입력 n개를 받는 예제에서는 첫 숫자 n을 데이터 값으로 합하지 않는지 확인한다. 토큰 목록 data의 길이 n+1,값 목록의 길이 n이라는 관계를 쓴다. 공백이 의미인 문자열 입력에서는 split이 원래 정보를 지운다는 점도 점검한다. 출력 마지막 줄바꿈은 요구 형식에 맞추되 디버그 문구를 넣지 않는다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 017회차 | 02-02 원리ㅍ

조건·반복·입출력·코드 추적 · 과목 17/80회차 · 기록 묶음 04의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

강화 응용: 조건을 바꾸어 다시 증명하기

루프를 추적할 때 continue와 break를 별도로 표시한다. 아래 코드는 Python3이며 답은 7이다.

```
s = 0
for x in [1, 2, 3, 4]:
    if x == 3:
        continue
    s += x
print(s)
```

continue는 현재 회차의 나머지를 건너뛰고 다음 x로 이동한다. break로 바꾸면 3에서 전체 루프를 끝내어 답은 3이다. Python의 for-else는 한 번도 실행 안 된 루프에도 실행될 수 있고, break 없이 정상 종료한 때 실행된다. '마지막 원소에서 조건을 발견했으므로 else가 실행된다'는 주장은 break가 있으면 틀린다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 018회차 | 02-02 예제·적용

조건·반복·입출력·코드 추적 · 과목 18/80회차 · 기록 묶음 04의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

입력의 형식도 알고리즘 계약이다. C++의 `while (cin >> x)`는 추출에 성공한 값만 처리한다. 입력이 `2 5 stop 9`라면 2와 5를 처리하고 `stop`에서 실패 상태가 되어 종료하며 9를 처리하지 않는다. 중간에 잘못된 토큰이 들어와도 알아서 건너뛰는 루프가 아니다. 전체 입력 유효성 보장이 없으면 실패를 어떻게 보고할지 별도 설계한다.

n=0 처리에서 최대값을 입력 첫 원소로 초기화하는 구현은 빈 배열 계약을 먼저 정해야 한다. 문제에서 n≥1이면 단순화할 수 있지만 일반 재사용 함수는 예외·없음 반환 등 정책을 명시한다. 입력과 출력 형식까지 포함한 작은 경계 사례를 직접 실행해야 코드가 완결된다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 019회차 | 02-02 직접 문제

조건·반복·입출력·코드 추적 · 과목 19/80회차 · 기록 묶음 04의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q28

Python3에서 s=0; for x in [1,2,3,4]: 내부에서 x==3이면 continue, 그 외 s+=x를 수행한다. 최종 s는?

1. 3

2. 7

3. 6

4. 8

내 선택·근거: _____

B02Q29

Python3의 for-else에서 else가 실행되는 조건으로 옳은 것은? 루프 본문에서 다른 예외/return은 없다고 가정한다.

1. 반복 본문을 한 번 이상 수행한 때만 실행된다.

2. 반복 중 break로 종료하지 않고 정상 종료한 때다.

3. break가 마지막 회차에서 발생하면 실행된다.

4. continue가 한 번이라도 있으면 실행되지 않는다.

내 선택·근거: _____

B02Q30

C++17에서 long long x,sum=0; while(cin>>x) sum+=x;를 수행한다. 입력 토큰은 2 5 stop 9이며 앞의 유효 덧셈은 범위 안이다. sum과 종료 이유는?

1. 16, 문자열 stop은 자동으로 무시한다.

2. 0, 파일에 잘못된 토큰이 있으면 이전 합도 취소한다.

3. 7, stop에서 정수 추출이 실패하여 반복을 끝낸다.

4. 9, 실패 뒤 마지막 숫자만 자동 복구한다.

내 선택·근거: _____

W02Q02

Python3에서 s=0으로 시작해 [1,2,3,4]를 순회하며 x==3이면 continue, 그 외 s+=x 한다. 각 회차의 s와 최종값을 적어라. continue를 break로 바꾼 결과, 빈 목록의 for-else 동작도 설명하라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 020회차 | 02-02 해설·재현

조건·반복·입출력·코드 추적 · 과목 20/80회차 · 기록 묶음 04의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q28

정답: 2번

풀이와 보기별 판정

「3」 - 오답. continue를 break로 바꾸면 1+2에서 종료하여 3이다.

「6」 - 오답. 3을 더하고 4를 빼는 잘못된 추적이다.

「7」 - 정답. 3 회차만 건너뛰어 1+2+4=7이다.

「8」 - 오답. 정상 회차 합은 7이며 한 회차를 중복 계산한 값이다.

B02Q29

정답: 2번

풀이와 보기별 판정

「반복 중 break로 종료하지 않고 정상 종료한 때다.」 - 정답. 반복 대상이 빈 경우도 break 없이 정상 종료하므로 else가 실행된다.

「반복 본문을 한 번 이상 수행한 때만 실행된다.」 - 오답. 빈 반복에도 else는 실행될 수 있다.

「break가 마지막 회차에서 발생하면 실행된다.」 - 오답. break는 위치와 무관하게 else를 생략한다.

「continue가 한 번이라도 있으면 실행되지 않는다.」 - 오답. continue는 루프 전체의 break 종료가 아니다.

B02Q30

정답: 3번

풀이와 보기별 판정

「16, 문자열 stop은 자동으로 무시한다.」- 오답. 추출 연산은 숫자 아닌 토큰을 자동으로 건너뛰지 않는다.

「7, stop에서 정수 추출이 실패하여 반복을 끝낸다.」- 정답. 2와 5만 성공하고 세 번째 추출이 실패해 조건이 false다.

「0, 파일에 잘못된 토큰이 있으면 이전 합도 취소한다.」- 오답. 이미 수행한 정상 덧셈은 취소되지 않는다.

「9, 실패 뒤 마지막 숫자만 자동 복구한다.」- 오답. 실패 상태를 해제하고 토큰을 처리하는 로직이 없다.

W02Q02

Python3에서 s=0으로 시작해 [1,2,3,4]를 순회하며 x==3이면 continue, 그 외 s+=x 한다. 각 회차의 s와 최종값을 적어라. continue를 break로 바꾼 결과, 빈 목록의 for-else 동작도 설명하라.

모범답안

x=1 후 s=1, x=2 후 3, x=3은 건너뛰어 3, x=4 후 7이다. continue는 현재 회차만 건너뛰므로 최종 7이다. break로 바꾸면 x=3에서 전체 루프 종료하여 최종 3이다. Python for-else에서 빈 목록은 break 없이 정상 종료하므로 else를 실행한다. else는 본문 수행 횟수보다 break로 종료했는지에 달려 있다.

10점 자가 채점 기준

- 네 회차 s 추적: 4점
- 최종 7과 continue 의미: 2점
- break 최종 3과 종료 범위: 2점
- 빈 목록 else 실행과 근거: 2점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 021회차 | 02-03 원리 I

함수·전달 방식·스코프·재귀 · 과목 21/80회차 · 기록 묶음 05의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

함수의 계약을 적기

함수는 입력 계약과 결과 계약을 갖는다. 입력을 변경하는지, 오류일 때 무엇을 하는지, 빈 입력을 허용하는지 적으면 구현과 호출자가 같은 기대를 갖는다. "함수를 호출하면 인자가 그대로 전달된다" 대신 값 복사·객체 공유·참조 바인딩을 구별한다.

1. C++ 값 전달과 참조 전달

C++에서 int x를 매개변수로 받으면 기본적으로 호출 시 값이 복사된다. int&는 기존 객체에 대한 참조이고 수정이 호출자 객체에 반영된다. const vector<int>&는 큰 컨테이너를 불필요하게 복사하지 않고 읽을 때 흔히 사용한다. const는 그 참조를 통한 변경을 제한하지만 모든 외부 경로에서 객체가 영원히 불변이라는 뜻은 아니다.

예제 A:

```
void by_value(int x) { x += 10; }
void by_ref(int& x) { x += 10; }
int n = 5;
by_value(n); // n == 5
by_ref(n); // n == 15
```

by_value의 지역 x와 n은 다른 객체다. by_ref의 x는 n을 가리키는 참조다. 포인터를 값으로 전달해도 포인터가 가리키는 객체를 수정할 수 있다. 포인터 변수 자체를 다른 주소로 바꾸는 일과 그 주소의 원소를 바꾸는 일을 나눠 그린다.

2. Python의 객체 공유와 재바인딩

Python 함수의 매개변수 이름은 전달된 객체에 연결된다. 이름을 새 객체에 다시 연결하는 일은 호출자의 이름 연결을 바꾸지 않지만 공유 중인 변경 가능 객체 자체를 수정하면 호출자에게도 보인다. 이를 C++의 int& 참조 매개변수와 완전히 같은 의미로 부르면 혼동한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 022회차 | 02-03 원리표

함수·전달 방식·스코프·재귀 · 과목 22/80회차 · 기록 묶음 05의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 B:

```
def change(xs):
    xs.append(3)
    xs = [9]
    return xs
a = [1, 2]
b = change(a)
print(a)
print(b)
```

처음 xs와 a가 같은 목록을 가리킨다. append 후 그 객체가[1,2,3]이 된다. xs=[9]는 함수 안 이름을 새 목록으로 연결하고 반환한다. 출력 a는[1,2,3], b는[9]다. 매개변수에 대입했다고 호출자 이름까지 새 목록으로 바뀌지는 않는다. C#의 참조 형 매개변수를 일반 값 전달한 상황도 이 두 종류의 변경을 구별해서 읽을 수 있다. C# ref/out은 별도 규칙이다.

3. 기본 인자와 스코프

Python의 기본 인자 표현식은 함수 정의 때 평가된다. 변경 가능한 목록을 기본값으로 두면 호출끼리 공유할 수 있다.

```
def add_bad(x, items=[]):
    items.append(x)
    return items
```

연속해서 add_bad(1),add_bad(2)를 호출하면 같은 기본 목록이[1]에서[1,2]로 변한다. 각 호출마다 새 목록이 필요한 의도라면 기본값 None을 두고 함수 안에서 새 목록을 만든다. 기본 목록을 의도적으로 공유하는 설계가 언제나 문법 오류라는 뜻은 아니다.

지역 이름과 바깥 이름의 범위를 구별한다. Python 함수에서 이름에 대입하면 기본적으로 지역 변수로 취급하므로 바깥 이름을 읽으려던 코드가 UnboundLocalError를 낼 수 있다. global·nonlocal은 바인딩 대상을 명시할 때 사용한다. C++의 블록 스코프, Java·C#의 변수 범위와 같은 이름이 등장해도 세부 규칙은 언어마다 확인한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 023회차 | 02-03 예제·적용

함수·전달 방식·스코프·재귀 · 과목 23/80회차 · 기록 묶음 05의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

4. 재귀의 기저 조건과 진전

재귀는 함수가 같은 함수를 직접 또는 간접 호출하는 구조다. 기저 조건이 있고 입력이 그 조건으로 가까워져야 한다. factorial(n)=n×factorial(n-1), factorial(0)=1은 n>=0에서 정의된다. 음수를 받아 n-1로 계속 내려가면0에 도달하지 않는다.

예제 C: fact(3)는 fact(2),fact(1),fact(0)을 호출하고 반환하며1→1→2→6을 만든다. 호출 깊이가4로 활성 정보가 쌓인다. 재귀 표현이 짧다고 추가 공간이0인 것은 아니다. Python 기본 구현은 일반적인 꼬리 호출 최적화를 보장하지 않으므로 깊이가 큰 입력에는 반복이나 명시적 스택이 필요할 수 있다.

5. 중복 부분 문제와 비용

fib(n)=fib(n-1)+fib(n-2)를 그대로 구현하면 같은 fib(3) 등을 여러 경로에서 계산한다. memo에 상태별 답을 저장하면 계산한 상태를 다시 쓰고 n에 비례한 상태만 계산한다. "재귀이면 느리다"가 아니라 중복 상태와 호출 비용을 살핀다. 이진 탐색 재귀는 구간이 반으로 줄어 호출 깊이가 O(log n)이고, 한 원소씩 줄이는 재귀는 O(n)일 수 있다.

함수 오버로딩은 같은 이름에 서로 다른 매개변수 형태를 두어 선택하는 기능이다. C++·Java·C#은 오버로딩 규칙을 제공하지만 Python에서는 같은 이름으로 def를 다시 하면 일반적으로 앞의 이름 바인딩을 새 함수로 덮는다. 동적인 인자 검사나 singledispatch 같은 기능과 문법적 오버로딩을 구별한다.

직접 연습·완료 기준

목록 매개변수에서 append와 xs=[...]를 각각 적용한 그림을 그린다. 기본 인자 목록의 공유를 None 패턴으로 수정한다. fact(0)=1, fact(4)=24를 추적하고 호출 깊이를 적는다. "입력 자체를 바꾸는지"와 "변수 이름을 새 객체로 연결하는지"를 말할 수 있으면 완료한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 024회차 | 02-03 직접 문제

함수·전달 방식·스코프·재귀 · 과목 24/80회차 · 기록 묶음 05의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q07

C++에서 void f(int x){x=9;}이고 int n=3;f(n);을 수행했다. n은?

1. 9

2. 0

3. 3

4. 정의되지 않은 동작

내 선택·근거: _____

B02Q08

Python 함수가 매개변수 xs에 append(3)한 뒤 xs=[9]를 반환한다. a=[1,2]를 전달하고 반환값을 b에 저장할 때 a,b는?

1. [9],[9]

2. [1,2,3],[9]

3. [1,2],[9]

4. [1,2,3],[1,2,3]

내 선택·근거: _____

B02Q09

Python 함수 기본 인자로 items=[]를 두고 매 호출 append한다. 옳은 설명은?

1. 매 호출 빈 목록이 자동으로 새로 생긴다.

2. 목록 기본 인자는 문법 오류다.

3. 기본 목록이 함수 정의 때 만들어져 호출끼리 공유될 수 있다.

4. 첫 호출 이후 함수가 사라진다.

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 025회차 | 02-03 해설·재현

함수·전달 방식·스코프·재귀 · 과목 25/80회차 · 기록 묶음 05의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q07

정답: 3번

int 값이 지역 매개변수로 복사되어 호출자 n은3이다. ①는 int&처럼 참조 전달을 가정했고, ②은 임의 초기화 규칙을 적용했고, ④는 정상적인 값 전달을 UB와 혼동했다.

B02Q08

정답: 2번

append는 공유 목록 자체를 바꾸고 xs=[9]는 지역 이름을 새 목록에 연결한다. a는[1,2,3],b는[9]다. ①은 지역 재바인딩이 호출자까지 바뀐다는 오류, ③는 공유 객체 변경 누락, ④는 반환값 재바인딩을 무시했다.

B02Q09

정답: 3번

기본 인자 평가 시점이 함수 정의 시점이므로 변경 목록이 공유될 수 있다. ①은 흔한 오해, ②은 문법적으로 허용되는 기능을 오류로 단정, ④는 함수 이름 바인딩과 무관한 주장이다. 새 목록이 필요하면 None 패턴을 쓴다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 026회차 | 02-03 원리 I

함수·전달 방식·스코프·재귀 · 과목 26/80회차 · 기록 묶음 06의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 실습: 새 기본 목록 만들기

각 호출마다 독립적인 목록이 필요하면 다음과 같이 쓴다.

```
def add_safe(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
a = add_safe(1)
b = add_safe(2)
print(a, b)
```

출력은[1] [2]다. 반대로 호출자가 목록을 직접 전달하면 그 목록을 수정하는 계약이다. "None으로 바꾸면 어떤 경우도 입력 변경이 없다"는 뜻은 아니다.

재귀의 작은 검증은 호출·반환 표로 한다. fact(4)→fact(3)→fact(2)→fact(1)→fact(0),반환1→1→2→6→24다. 일반적인 최적화가 없는 호출 모델에서 최대 활성 호출은5개다. n=0에서도 답1을 내는 초깃값,음수를 허용하지 않는 입력 조건을 함께 적는다. 기저 조건을 if n==1로만 쓰면0에서 종료하지 못하는 문제를 만들 수 있다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 027회차 | 02-03 원리ㅍ

함수·전달 방식·스코프·재귀 · 과목 27/80회차 · 기록 묶음 06의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

강화 응용: 조건을 바꾸어 다시 증명하기

Python 기본 인수는 함수가 호출될 때마다 새로 평가되는 것이 아니다. 다음 예는 호출 사이 목록을 공유한다.

```
def append_bad(x, bag=[]):
    bag.append(x)
    return bag
```

첫 append_bad(1)의 결과는 [1], 두 번째 append_bad(2)는 같은 객체 [1,2]다. 결과 참조를 저장해 두었다면 첫 결과도 나중에 [1,2]로 보인다. 호출별 독립 목록이 목적이면 bag=None을 기본값으로 하고 함수 안에서 None일 때 []를 만든다. None이 유효한 업무 값이면 별도 sentinel을 써야 한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 028회차 | 02-03 예제·적용

함수·전달 방식·스코프·재귀 · 과목 28/80회차 · 기록 묶음 06의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

클로저의 늦은 바인딩도 유사한 함정이다. fs=[lambda: i for i in range(3)]에서 작성 후 호출한 결과는 [2,2,2]다. 함수가 숫자를 복사해 저장한 것이 아니라 나중에 같은 셀의 i를 읽는다. lambda i=i: i처럼 정의 시 기본 인수로 값을 잡으면 [0,1,2]가 된다. 반복 안에서 함수를 즉시 호출하는 사례와 반복 후 호출하는 사례를 구분한다.

꼬리 위치 재귀라도 Python이나 C++17이 호출 스택을 항상 제거해 준다고 계약하지 않는다. 높이 n인 재귀 탐색은 일반적으로 O(n) 호출 공간을 계산한다. 반복으로 바꾸면 변수 몇 개로 처리 가능한 문제인지, 명시적 스택이 필요한 문제인지에 따라 공간이 달라진다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 029회차 | 02-03 직접 문제

함수·전달 방식·스코프·재귀 · 과목 29/80회차 · 기록 묶음 06의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q31

Python3: def f(x, bag=[]): bag.append(x); return bag 를 두 번 호출한다. 첫 결과를 a=f(1), 둘째를 b=f(2)로 저장했을 때 a,b와 객체 동일성은?

1. a=[1], b=[2], 서로 다른 객체다.

2. a=[1,2], b=[1,2], 같은 객체다.

3. a=[1], b=[1,2], 서로 다른 객체다.

4. 둘째 호출은 기본 인수가 없어 오류다.

내 선택·근거: _____

B02Q32

Python3에서 fs=[lambda: i for i in range(3)] 작성 후 [f() for f in fs]를 평가한다. 결과는?

1. [0,1,2]

2. [0,0,0]

3. [1,1,1]

4. [2,2,2]

내 선택·근거: _____

B02Q33

Python3에서 호출 깊이가 n인 꼬리 위치 재귀를 사용한다. 별도 최적화 보장이 없는 실행 모델의 추가 호출 공간을 가장 타이트하게 평가하면?

1. Θ(1)

2. Θ(log n)

3. Θ(n^2)

4. Θ(n)

내 선택·근거: _____

W02Q03

Python 함수 def f(x, bag=[]): bag.append(x); return bag 의 상태 공유 문제를 설명하고, 호출마다 새 목록을 만드는 수정 코드를 제시하라. f(1),f(2)의 수정 전·후 결과도 적어라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 030회차 | 02-03 해설·재현

함수·전달 방식·스코프·재귀 · 과목 30/80회차 · 기록 묶음 06의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q31

정답: 2번

풀이와 보기별 판정

「a=[1], b=[2], 서로 다른 객체다.」 - 오답. 기본 목록은 정의 시 한 번 생성되어 공유된다.

「a=[1], b=[1,2], 서로 다른 객체다.」 - 오답. a도 공유 목록을 가리켜 둘째 수정이 보인다.

「a=[1,2], b=[1,2], 같은 객체다.」 - 정답. 두 호출 모두 같은 기본 목록을 사용하므로 a is b가 true다.

「둘째 호출은 기본 인수가 없어 오류다.」 - 오답. 기본 인수는 여러 호출에서 사용할 수 있다.

B02Q32

정답: 4번

풀이와 보기별 판정

「[0,1,2]」 - 오답. 이 값은 lambda i=i: i처럼 정의 시 값을 잡아야 얻는다.

「[0,0,0]」 - 오답. 셀의 최종 i는 0이 아니다.

「[1,1,1]」 - 오답. 셀의 최종 i는 1이 아니다.

「[2,2,2]」 - 정답. 각 함수가 동일한 i 셀을 나중에 읽으며 마지막 값은 2다.

B02Q33

정답: 4번

풀이와 보기별 판정

「Θ(n)」- 정답. 호출마다 프레임이 남으며 깊이가 n이다.

「Θ(1)」- 오답. Python이 꼬리 호출을 자동 제거한다고 보장할 수 없다.

「Θ(log n)」- 오답. 깊이가 log n이라는 조건이 아니다.

「Θ(n^2)」- 오답. 프레임 하나당 고정 크기면 깊이와 선형 관계다.

W02Q03

Python 함수 def f(x, bag=[]): bag.append(x); return bag 의 상태 공유 문제를 설명하고, 호출마다 새 목록을 만드는 수정 코드를 제시하라. f(1),f(2)의 수정 전·후 결과도 적어라.

모범답안

수정 전 bag=[]는 함수 정의 시 한 번 평가되므로 f(1) 후 같은 객체에 [1], f(2) 후 [1,2]가 된다. 앞 결과 참조도 [1,2]로 바뀐다. 수정 코드는 다음과 같다. 초기화만 if 내부에 두고 append와 return은 if 밖에 둔다.

```
def f(x, bag=None):
    if bag is None:
        bag = []
    bag.append(x)
    return bag
```

인수 생략 호출 f(1),f(2)는 서로 다른 [1],[2]를 돌려준다. 호출자가 목록을 명시하면 그 목록을 수정한다는 계약은 유지한다. None 자체가 유효 입력이면 별도 sentinel 정책이 필요하다.

10점 자가 채점 기준

- 정의 시 한 번 생성과 참조 공유: 3점
- 수정 전 두 결과와 앞 참조 변화: 2점
- None 검사 후 새 목록 생성 코드: 3점
- 수정 후 [1]/[2] 및 명시 목록 계약: 2점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 031회차 | 02-04 원리 I

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 31/80회차 · 기록 묶음 07의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

변수와 객체를 따로 그리기

메모리 문제는 이름이 몇 개 있는지보다 실제 객체가 몇 개 있고 누가 그 객체를 참조하는지가 중요하다. 변수→객체 상자를 그려 대입·복사·변경·수명 종료를 추적한다. 언어 런타임의 구현을 학습 모델과 혼동하지 않는다.

1. 자동 수명과 동적 수명

C++ 지역 객체는 일반적으로 블록을 벗어나면 수명이 끝나고 소멸자가 호출된다. 동적으로 할당한 객체의 수명은 별도로 관리해야 한다. 스마트 포인터와 RAII는 수명과 자원 소유를 객체 구조에 연결하는 설계다. 함수의 지역 변수 주소를 반환해 수명이 끝난 객체를 사용하면 유효한 접근을 보장할 수 없다.

"지역 변수는 항상 물리적 스택, new는 무조건 힙"이라는 단순 문장은 최적화·표현·런타임 구현까지 정확히 설명하지 못한다. 시험의 일반적인 실행 모델에서는 호출 스택과 동적 저장 공간을 구별하되, C++에서 중요한 언어 규칙은 저장 기간과 객체 수명이다. C# 값 형이 항상 스택에만 존재하는 것도 아니다. 클래스 객체의 필드나 배열 원소로 저장된 값 형을 생각하면 반례를 만들 수 있다.

2. 대입과 얕은 복사

Python에서 a=[1,2], b=a는 새 목록을 만들지 않는다. 두 이름이 같은 목록을 가리킨다. b.append(3)이 a에서도 보인다. b=a.copy()는 바깥 목록을 새로 만들지만 원소 참조를 복사한다. 원소가 변경 불가능한 정수이면 이런 얕은 복사가 독립된 값 목록처럼 보일 수 있지만 중첩된 목록이면 차이가 드러난다.

예제 A:

```
a = [[1], [2]]
b = a.copy()
b[0].append(9)
b.append([3])
print(a)
print(b)
```

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 032회차 | 02-04 원리ㅍ

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 32/80회차 · 기록 묶음 07의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

두 바깥 목록은 다르지만 첫 내부 목록을 공유한다. a=[[1,9],[2]], b=[[1,9],[2],[3]]다. 내부 변경은 공유되어 보이고 바깥 append는 b에만 보인다. "얕은 복사는 모든 것이 공유"도 "copy했으니 완전 독립"도 틀린 설명이다.

3. 깊은 복사와 예외

깊은 복사는 중첩 객체까지 재귀적으로 복사하는 접근이다. Python copy.deepcopy는 순환·이미 처리한 객체를 메모하여 적절한 복사 구조를 만들며 사용자 정의 객체는 복사 동작을 제어할 수 있다. 파일·소켓·외부 자원을 "완전히 복사하면 항상 독립"이라는 말은 현실적인 자원 의미까지 보장하지 않는다. 여기서는 정수와 목록으로 된 데이터 구조를 대상으로 연습한다.

예제 B: a=[[1],[2]], c=deepcopy(a) 후 c[0].append(9)이면 a는[[1],[2]], c는[[1,9],[2]]이다. 같은 입력에 얕은 복사와 깊은 복사를 비교하는 표를 만든다. C++ vector<vector<int>>를 값 복사하면 각 vector가 보유한 정수 원소들을 복사하는 일반적인 값 의미를 갖는다. 그러나 vector<shared_ptr<T>>를 복사하면 포인터 소유 정보가 복사되어 T 객체를 공유할 수 있다. 컨테이너 종류만 보고 "항상 깊은 복사"라고 하지 않는다.

4. 2차원 목록 초기화의 함정

Python grid=[[0]*3]*2는 같은 내부 목록을 두 번 참조한다. grid[0][1]=7이면 두 행 모두[0,7,0]으로 보인다. 각 행을 독립적으로 만들려면 [[0]*3 for _ in range(2)]로 작성한다. 바깥 반복이 목록 객체를 새로 만드는지를 확인한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 033회차 | 02-04 예제·적용

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 33/80회차 · 기록 묶음 07의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

손풀이 예제 C: 처음 내부 객체 R=[0,0,0], grid=[R,R]이다. R의 인덱스1을 바꾸면 grid의 두 경로 모두 같은 변경을 본다. 원소 대입을 하지 않고 grid[0]=[1,1,1]처럼 첫 행 참조 자체를 바꾸면 두 번째 행은 R을 유지한다. 객체 변경과 참조 교체를 나누어 이해한다.

5. 변경 불가능성과 동일성

Python str·int·tuple의 바깥 구조는 불변이다. tuple 안에 목록을 넣으면 목록 자체는 변경될 수 있다. tuple=([1],) 후 tuple[0].append(2)는 내부 목록 변경이며 tuple 항목 재대입과 다르다. C# string도 불변이며 새 문자열을 만드는 연산과 원본 문자 배열 수정은 구별한다.

Python ==는 값 비교를 호출하고 is는 동일 객체인지 검사한다. 작은 정수 캐싱이나 문자열 interning은 구현 세부이므로 숫자 두 개의 is 결과를 일반 시험 규칙으로 암기하지 않는다. None 확인은 x is None처럼 의미가 명확한 동일성 검사다. 값이 같은지를 원하면 ==를 쓴다.

6. GC와 자원 정리

가비지 컬렉션은 더 이상 사용 가능한 참조가 없는 관리 메모리의 회수를 돕는다. 파일·DB연결의 즉시 해제 시점을 무조건 보장하는 의미는 아니다. C# using, Python with, C++ RAII 등은 자원 사용 범위를 명시하는 패턴으로 연결된다. 메모리 누수는 참조를 계속 유지해 필요 없는 객체가 회수되지 않는 경우에도 생길 수 있다.

완료 기준

종이에 바깥 컨테이너2개와 공유 내부 컨테이너1개를 그려 예제 A를 재현한다. 얕은 복사·깊은 복사·대입 세 가지를 구분하고, C++의 수명 종료 후 포인터 접근을 정해진 출력으로 단정하지 않으면 완료한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 034회차 | 02-04 직접 문제

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 34/80회차 · 기록 묶음 07의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q10

Python a=[[1],[2]],b=a.copy() 후 b[0].append(9)와 b.append([3])을 수행했다. a는?

1. [[1,9],[2]]

2. [[1],[2]]

3. [[1,9],[2],[3]]

4. [[9],[3]]

내 선택·근거: _____

B02Q11

Python grid=[[0]*3]*2;grid[0][1]=7 후 두 행은?

1. [0,7,0]과[0,0,0]

2. [7,0,0]과[7,0,0]

3. 오류 때문에 어떤 행도 없다.

4. [0,7,0]과[0,7,0]

내 선택·근거: _____

B02Q12

메모리·수명에 대한 설명으로 맞는 것은?

1. C# 값 형은 항상 물리적 스택에만 있다.

2. GC가 있으므로 파일을 명시적으로 닫을 필요는 전혀 없다.

3. C++ 지역 객체 수명이 끝난 후 그 객체의 주소로 접근한 결과를 특정 값으로 보장할 수 없다.

4. Python에서 a=b는 언제나 깊은 복사다.

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 035회차 | 02-04 해설·재현

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 35/80회차 · 기록 묶음 07의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q10

정답: 1번

바깥 목록은 별도이나 첫 내부 목록을 공유해[1,9]가 보인다. 바깥 append는 a에 반영되지 않는다. ②은 깊은 복사처럼 읽었고, ③은 바깥까지 공유하는 대입처럼 읽었고, ④는 append를 교체 연산으로 혼동했다.

B02Q11

정답: 4번

두 행이 동일한 내부 목록을 참조해 같은 변경을 본다. ①은 독립 행 comprehension 결과와 혼동하고, ②은 인덱스1을0으로 잘못 읽었다. ③는 유효한 리스트 수정이므로 틀리다.

B02Q12

정답: 3번

수명 종료 후 유효 객체 사용을 보장하지 않는다. ①은 객체 필드·배열 안의 값 형 등 저장 상황을 무시한다. ②는 관리 메모리 회수와 파일 자원 해제를 혼동한다. ④는 두 이름의 객체 공유인 대입을 복사로 잘못 읽었다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 036회차 | 02-04 원리 I

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 36/80회차 · 기록 묶음 08의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 실습: 얕은 복사와 깊은 복사를 같은 입력으로 비교

한 코드에서 두 복사의 차이를 검증한다.

```
from copy import deepcopy
original = [[1], [2]]
shallow = original.copy()
deep = deepcopy(original)
shallow[0].append(9)
deep[1].append(8)
print(original)
print(shallow)
print(deep)
```

출력은 original=[[1,9],[2]],shallow=[[1,9],[2]],deep=[[1],[2,8]]다. deep은 shallow 수정 전에 복사했으므로 첫 내부 목록에9가 없다. 복사 시점까지 결과에 영향을 준다. "deep에는 항상 원본의 현재 값이 따라온다"는 오해를 제거한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 037회차 | 02-04 원리ㅍ

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 37/80회차 · 기록 묶음 08의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

순환 구조 a=[];a.append(a)는 깊은 복사에서 무한히 새 하위 객체를 만들지 않도록 메모 관리가 필요한 이유를 보여준다. 이 구조의 자세한 repr를 시험 정답으로 외우기보다,같은 객체를 다시 방문한다는 그림을 그린다. 값 복사·공유 소유·고유 소유를 구별하는 습관이 더 중요하다.

강화 응용: 조건을 바꾸어 다시 증명하기

2차원 목록의 겉 복사와 내부 객체 복사를 손으로 그린다. a=[[1],[2]], b=a.copy()이면 a와 b는 별도 바깥 목록이지만 a[0] is b[0]은 true다. b[0].append(9)는 a에도 보여 a=[[1,9],[2]]가 된다. 이후 b[1]=[7]은 b의 두 번째 참조만 교체하므로 a[1]=[2]는 그대로다. '얕은 복사는 언제나 원본을 바꾼다'도 지나친 일반화다. 내부 객체 수정과 바깥 참조 교체를 구분한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 038회차 | 02-04 예제·적용

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 38/80회차 · 기록 묶음 08의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

C++ vector는 재할당이 발생하면 기존 원소를 가리키던 포인터·참조·반복자가 무효화된다. size=capacity=3에서 push_back하여 재할당했다면 이전 v[0] 참조를 읽으면 안 된다. 값을 복사한 독립 변수는 살아 있지만 기존 주소를 저장한 변수는 그렇지 않다. reserve를 호출했다는 사실만으로 모든 이후 삽입이 안전한 것은 아니며 실제 새 size가 capacity를 넘는지 확인한다.

소유권과 관찰용 주소도 구분한다. unique_ptr<T>는 하나의 소유자가 자원을 해제하며 복사는 금지되고 이동으로 소유권을 옮긴다. raw pointer를 복사했다고 소유권이 자동 이전되거나 대상 수명이 늘어나지는 않는다. 스택 지역 변수의 주소를 반환한 뒤 그 함수를 벗어나면 대상 수명이 끝나므로 이후 역참조가 안전하지 않다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 039회차 | 02-04 직접 문제

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 39/80회차 · 기록 묶음 08의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q34

Python3: a=[[1],[2]]; b=a.copy(); b[0].append(9); b[1]=[7] 를 수행했다. 최종 a,b는?

1. a=[[1],[2]], b=[[1,9],[7]]

2. a=[[1,9],[7]], b=[[1,9],[7]]

3. a=[[1,9],[2]], b=[[1,9],[7]]

4. a=[[1],[7]], b=[[1],[7]]

내 선택·근거: _____

B02Q35

C++17 vector<int> v의 size=capacity=3이다. int& r=v[0]을 얻은 뒤 push_back으로 재할당이 실제 발생했다. 옳은 설명은?

1. r은 무효화되어 이후 읽으면 안 된다.

2. r은 반드시 새 저장소의 v[0]을 자동으로 가리킨다.

3. 재할당은 끝 반복자만 무효화한다.

4. int형이므로 모든 참조가 항상 유효하다.

내 선택·근거: _____

B02Q36

C++17 unique_ptr<T>의 소유권 이전에 대한 옳은 설명은?

1. 일반 복사로 두 unique_ptr가 같은 객체를 함께 소유한다.

2. std::move를 이용한 이동으로 소유권을 옮기고 원래 unique_ptr는 소유를 잃는다.

3. raw pointer를 복사하면 unique_ptr 소유권도 자동으로 옮겨진다.

4. unique_ptr가 사라져도 대상은 절대 해제되지 않는다.

내 선택·근거: _____

W02Q04

a=[[1],[2]], b=a.copy()에서 b[0].append(9), b[1]=[7]을 수행한다. 객체 공유 관계와 최종 a,b를 설명하라. 독립된 내부 목록까지 필요한 경우의 방법과 일반 deepcopy의 한계도 적어라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 040회차 | 02-04 해설·재현

메모리·수명·참조·얕은 복사와 깊은 복사 · 과목 40/80회차 · 기록 묶음 08의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q34

정답: 3번

풀이와 보기별 판정

「a=[[1,9],[2]], b=[[1,9],[7]]」- 정답. 첫 내부 목록은 공유 수정되고 두 번째 바깥 참조 교체는 b만 바꾼다.

「a=[[1],[2]], b=[[1,9],[7]]」- 오답. 얕은 복사는 내부 목록까지 복제하지 않는다.

「a=[[1,9],[7]], b=[[1,9],[7]]」- 오답. b[1] 교체는 별도 바깥 목록 a의 항목을 바꾸지 않는다.

「a=[[1],[7]], b=[[1],[7]]」- 오답. 첫 공유 내부 목록의 append를 빠뜨렸다.

B02Q35

정답: 1번

풀이와 보기별 판정

「r은 반드시 새 저장소의 v[0]을 자동으로 가리킨다.」- 오답. 참조는 재할당된 새 주소로 자동 재결합하지 않는다.

「r은 무효화되어 이후 읽으면 안 된다.」- 정답. 재할당은 기존 원소 주소에 연결된 포인터·참조·반복자를 무효화한다.

「재할당은 끝 반복자만 무효화한다.」- 오답. 끝 반복자 외에 모든 기존 원소 참조도 영향을 받는다.

「int형이므로 모든 참조가 항상 유효하다.」- 오답. 수명/주소 유효성은 원소가 단순 int라는 사실로 보장되지 않는다.

B02Q36

정답: 2번

풀이와 보기별 판정

「일반 복사로 두 unique_ptr가 같은 객체를 함께 소유한다.」 - 오답. unique_ptr는 독점 소유이며 일반 복사는 금지된다.

「raw pointer를 복사하면 unique_ptr 소유권도 자동으로 옮겨진다.」 - 오답. raw 주소 복사는 소유권을 전달하는 동작이 아니다.

「std::move를 이용한 이동으로 소유권을 옮기고 원래 unique_ptr는 소유를 잃는다.」 - 정답. 정상 이동 후 대상의 독점 소유자가 바뀌고 원래 포인터는 비워진다.

「unique_ptr가 사라져도 대상은 절대 해제되지 않는다.」 - 오답. 소유자가 소멸하면 사용자 정의 deleter 등 정책으로 자원을 해제한다.

W02Q04

a=[[1],[2]], b=a.copy()에서 b[0].append(9), b[1]=[7]을 수행한다. 객체 공유 관계와 최종 a,b를 설명하라. 독립된 내부 목록까지 필요한 경우의 방법과 일반 deepcopy의 한계도 적어라.

모범답안

a,b는 별도 바깥 목록이고 내부 [1],[2] 객체는 공유한다. append는 공유 첫 목록을 [1,9]로 바꾸고 b[1]=[7]은 b의 참조만 바꾼다. 최종 a=[[1,9],[2]], b=[[1,9],[7]]다. 단순 정수 목록 행이라면 [row.copy() for row in a]로 행까지 복제할 수 있다. 더 깊은 일반 객체 구조는 copy.deepcopy를 검토하지만 파일/연결/사용자 정의 객체 등 모든 외부 자원을 독립 복제한다고 보장하지 않는다.

10점 자가 채점 기준

- 바깥 별도/내부 공유 구조: 3점
- 두 연산 역할과 최종 값: 4점
- 행 복사 또는 적절한 deepcopy 방법: 2점
- 외부 자원/사용자 정의 객체 한계: 1점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 041회차 | 02-05 원리 I

문자열·배열·컨테이너와 반복자의 계약 · 과목 41/80회차 · 기록 묶음 09의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

인덱스는 구간 정의와 함께 사용한다

0 기반 인덱스의 유효 범위는 길이 n에서0..n-1이다. 빈 배열에는 유효 원소 인덱스가 없다. Python은 음수 인덱스를 끝 기준으로 해석하지만 C++ vector의 -1 접근은 그런 규칙이 아니다. Python의 슬라이스와 정수 인덱스가 경계를 처리하는 방식도 다르다.

1. 배열·슬라이스·문자열

Python a[1:4]는 인덱스1,2,3을 새 목록으로 만든다. 리스트 슬라이스는 원소 참조의 얕은 복사다. a[100]은 범위 밖이면 IndexError지만 a[:100]은 가능한 구간까지만 반환한다. slice가 무조건 O(1)이라는 주장은 일반 목록 복사 비용을 빠뜨린다.

손풀이 예제 A: a=[10,20,30,40,50], a[-1]=50, a[1:4]=[20,30,40], a[::2]=[10,30,50], a[::-1]=[50,40,30,20,10]. 마지막의 새 목록과 a.reverse()의 원본 변경은 구별한다. 문자열 s="ABCD"에서 s[1:3]="BC"지만 s[0]="Z"는 허용되지 않는다.

C++ vector의 []는 일반적으로 범위 검사를 제공하지 않고 at는 범위를 검사해 out_of_range를 낼 수 있다. 존재하지 않는 인덱스를 읽은 결과를 특정 숫자로 추측하지 않는다. C의 배열이 함수 인자로 전달될 때 일반적인 매개변수 선언은 포인터로 조정되므로 크기를 따로 전달해야 하는 상황이 많다. 함수 인자 a에서 sizeof(a)가 원래 배열 전체 크기를 알려준다고 일반화하지 않는다.

2. 문자의 저장 단위

ASCII는 기본7비트 부호 체계이고 Unicode는 문자에 코드 포인트를 부여하는 체계다. UTF-8은 가변 길이 인코딩이다. Python str의 길이와 UTF-8로 인코딩한 byte 길이는 다를 수 있다. 화면에 보이는 한 글자와 코드 포인트 하나도 결합 문자·이모지 때문에 항상 같지 않다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 042회차 | 02-05 원리ㅍ

문자열·배열·컨테이너와 반복자의 계약 · 과목 42/80회차 · 기록 묶음 09의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 B: s="가A"는 Python에서 코드 포인트 두 개라 len(s)=2, UTF-8에서 "가"3바이트+"A"1바이트이므로 len(s.encode("utf-8"))=4다. C++ std::string으로 UTF-8 바이트를 저장하면 size는 여기서4이고 한 바이트 인덱스 접근이 "한국어 글자 하나"를 반환하지 않는다. C# string은 UTF-16 코드 단위 모델이라 보조 평면 문자에서 Length와 표시 문자 수가 다를 수 있다. 본 교재의 단순 코드 출력 문제는 ASCII로 제한할 때 가정을 명시한다.

3. 집합·사전·맵

Python set은 중복 없는 집합이며 정렬 순서를 보장하지 않는다. dict는 키→값 관계이고 Python3.7부터 언어 수준의 삽입 순서 보장이 있다. 같은 키에 새 값을 넣으면 기존 값이 바뀐다. 없는 키를 []로 읽으면 KeyError, get으로 기본값을 제공하면 다른 정책을 구현한다.

예제 C: d={"A":2,"B":1};d["A"]=5;d["C"]=3이면 값은A5,B1,C3이다. 기존 A값을 바꾼 것이 새 A키를 추가한 것은 아니다. 빈 집합은 set(), {}는 빈 dict다. Python list는 일반적으로 해시 가능하지 않으므로 dict 키로 사용할 수 없다. tuple도 내부에 list를 포함하면 해시 가능성을 잃을 수 있다.

C++ map은 정렬 비교 순서에 따라 키를 관리하는 연관 컨테이너, unordered_map은 해시 기반 구조다. map과 dict가 항상 같은 순서·비용·삽입 부작용을 갖는다고 생각하지 않는다. Python Counter와 defaultdict를 써도 "값을 세는 연산"과 "처음 본 키 초기화"의 논리를 먼저 이해한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 043회차 | 02-05 예제·적용

문자열·배열·컨테이너와 반복자의 계약 · 과목 43/80회차 · 기록 묶음 09의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

4. 반복 중 수정과 무효화

컨테이너를 순회하면서 구조를 변경하면 방문 누락·반복자 무효화·오류가 생길 수 있다. 언어마다 규칙이 다르므로 안전한 기본 패턴은 삭제할 대상을 따로 모으거나 새 결과를 만드는 것이다. 무조건 어떤 수정도 불가능하다는 말 대신 구조 변경 종류와 반복자 계약을 읽는다.

예제 D: [1,2,3,4]를 순회하며 짝수를 지우려는 의도는 [x for x in a if x%2!=0]로 새 목록을 만들어[1,3]을 얻을 수 있다. 다른 이름과 원본을 공유 중이면 a의 재바인딩과 a[:]의 내용 교체는 효과가 다르므로 요구를 확인한다.

C++ vector의 재할당은 원소를 가리키는 반복자·포인터·참조를 무효화할 수 있다. erase는 삭제 위치 이후의 반복자 등에 영향을 준다. 연결 리스트에서는 삽입·삭제의 무효화 범위가 다르다. "STL 반복자는 전부 안전/전부 위험"으로 한 문장에 묶지 않는다.

5. 정렬·비교 함수의 전제

사용자 비교는 일관된 순서 관계를 제공해야 한다. C++ sort 비교에서 a<=b처럼 동일 값도 참으로 반환하면 strict weak ordering 조건을 깨뜨릴 수 있다. Python key 함수는 비교할 키를 만들며 key=lambda item:item[1]처럼 사용한다. 키가 같을 때 원래 순서가 유지되는 안정성은 정렬 라이브러리의 보장과 연결한다.

직접 연습·완료 기준

슬라이스 [l,r)의 길이 r-l과 UTF-8 바이트 길이를 구별한다. 컨테이너를 고를 때 인덱스 접근·존재 확인·순서·중복이라는 요구를 각각 적는다. a=b, a=b.copy(), a[:]=b가 객체 공유에 미치는 효과를 그림으로 말하면 다음 장으로 이동한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 044회차 | 02-05 직접 문제

문자열·배열·컨테이너와 반복자의 계약 · 과목 44/80회차 · 기록 묶음 09의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q13

Python a=[10,20,30,40,50]에서 a[1:4]는?

1. [10,20,30,40]

2. [20,30,40,50]

3. [30,40]

4. [20,30,40]

내 선택·근거: _____

B02Q14

Python s='가A'에서 len(s)와 len(s.encode('utf-8'))는?

1. 2,4

2. 2,2

3. 4,2

4. 4,4

내 선택·근거: _____

B02Q15

Python에서 빈 집합을 만드는 표현은?

1. {}

2. []

3. ()

4. set()

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 045회차 | 02-05 해설·재현

문자열·배열·컨테이너와 반복자의 계약 · 과목 45/80회차 · 기록 묶음 09의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q13

정답: 4번

시작1 포함·끝4 제외로 인덱스1,2,3이다. ①은 시작을0으로, ②은 끝을 포함으로, ③는 시작을2로 잘못 해석했다.

B02Q14

정답: 1번

코드 포인트가2개이고 UTF-8은 가3바이트+A1바이트로4바이트다. ②은 문자 수와 바이트 수를 같다고 보았고, ③은 두 단위를 반대로, ④는 str 길이를 바이트 길이로 해석했다. 이 문자열에는 결합 문자 등 추가 조건이 없다.

B02Q15

정답: 4번

set()가 빈 집합이다. ①은 빈 dict, ②는 빈 list, ③은 빈 tuple이다. 모두 비어 있다는 사실과 컨테이너 종류는 다르다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 046회차 | 02-05 원리 I

문자열·배열·컨테이너와 반복자의 계약 · 과목 46/80회차 · 기록 묶음 10의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 손풀이: 문자열 처리의 목적부터 정하기

문자열 " A B "에 strip()을 적용하면 "A B"이고 split()을 적용하면["A","B"]다. 앞뒤 공백 제거와 공백 기준 분리는 다른 연산이다. split(" ")처럼 구분자를 명시하면 연속 공백 처리도 달라질 수 있다. 로그 줄의 원문을 보존하는 문제와 토큰을 추출하는 문제를 구별한다.

C 문자열 char s[]="ABC"는 널까지 저장하지만 Python str "ABC"의 len은3이라는 문자 단위 계약이다. C++ std::string("ABC").size()는3이다. C의 sizeof(s)4와 C++ 문자열 길이3을 동일한 자료구조의 동일 연산처럼 비교하면 안 된다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 047회차 | 02-05 원리ㅍ

문자열·배열·컨테이너와 반복자의 계약 · 과목 47/80회차 · 기록 묶음 10의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

검색 값이 없을 때 Python list.index는 ValueError,dict[key]는 KeyError를 낼 수 있다. C++ find는 보통 end()를 반환한다. 원하는 실패 정책에 맞게 조건을 작성하고,end()를 역참조하지 않는다.

강화 응용: 조건을 바꾸어 다시 증명하기

문자열의 길이 단위를 확인한다. Python3 str인 '가A'의 len은 Unicode 코드 포인트 두 개를 세어 2다. 이 문자열을 UTF-8로 인코딩한 bytes의 길이는 '가' 3바이트와 'A' 1바이트를 더한 4다. C++ std::string에 같은 UTF-8 바이트열을 저장하면 size()는 바이트 수 4다. 코드 포인트 개수도 사용자가 보는 글자 묶음과 항상 같지는 않으므로 '화면 글자 수' 계약은 별개다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 048회차 | 02-05 예제·적용

문자열·배열·컨테이너와 반복자의 계약 · 과목 48/80회차 · 기록 묶음 10의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

C++ 정렬 비교자는 엄격한 순서 계약을 지켜야 한다. comp(a,b)를 a<=b로 쓰면 comp(x,x)=true여서 비반사성 조건을 위반한다. 오름차순 기본 비교는 a<b다. 비교자로 단순 a-b를 반환하는 것도 bool 변환 때문에 양·음 모두 true가 되고 정수 오버플로 위험까지 생길 수 있다. 비교자는 '첫 값이 앞서는가'를 bool로 반환해야 한다.

순회 중 목록의 길이나 위치를 바꾸는 코드는 별도 검토한다. Python에서 a=[1,2,3,4]; for x in a: a.remove(x)를 수행하면 1을 지운 뒤 다음 인덱스가 현재 3을 읽고, 3을 지운 뒤 끝나므로 [2,4]가 남는다. 원래 항목을 모두 지우려면 clear()를 쓰거나, 조건 필터링이면 새 목록을 만들어 교체한다. 라이브러리가 컴파일/실행을 허용한다는 사실은 의도대로 모든 원소를 검사했다는 보장이 아니다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 049회차 | 02-05 직접 문제

문자열·배열·컨테이너와 반복자의 계약 · 과목 49/80회차 · 기록 묶음 10의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q37

Python3 s="가A"에서 len(s), len(s.encode("utf-8"))의 값은?

1. (2,2)

2. (4,4)

3. (2,4)

4. (4,2)

내 선택·근거: _____

B02Q38

C++17 std::sort의 정수 오름차순 비교자로 올바른 엄격 비교는?

1. return a <= b;

2. return a < b;

3. return a - b;

4. return true;

내 선택·근거: _____

B02Q39

Python3에서 a=[1,2,3,4]를 만든 뒤 for x in a: a.remove(x)를 실행한다. 최종 a는?

1. []

2. [1,3]

3. [2,4]

4. [4]

내 선택·근거: _____

W02Q05

Python3 str "가A", UTF-8 bytes, 같은 UTF-8 데이터를 담은 C++ std::string의 길이를 각각 적어라. 코드 포인트 수/바이트 수/사용자에게 보이는 문자 수를 같은 개념으로 봐도 되는지 설명하라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 050회차 | 02-05 해설·재현

문자열·배열·컨테이너와 반복자의 계약 · 과목 50/80회차 · 기록 묶음 10의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q37

정답: 3번

풀이와 보기별 판정

「(2,2)」 - 오답. 한글 가는 UTF-8 3바이트이므로 bytes 길이는 4다.

「(4,4)」 - 오답. str 길이는 바이트 수가 아니라 코드 포인트 수 2다.

「(4,2)」 - 오답. 두 길이의 단위를 뒤집었다.

「(2,4)」 - 정답. 코드 포인트 2개와 UTF-8 바이트 3+1=4를 구분한다.

B02Q38

정답: 2번

풀이와 보기별 판정

「return a < b;」 - 정답. 같은 값에서는 false이고 일관된 엄격 순서를 제공한다.

「return a <= b;」 - 오답. comp(x,x)가 true여서 엄격 비교 계약을 위반한다.

「return a - b;」 - 오답. 음수와 양수 차이 모두 bool true일 수 있고 뺄셈 오버플로 위험도 있다.

「return true;」 - 오답. 모든 값이 서로 앞선다고 하여 비반사성/비대칭성을 위반한다.

B02Q39

정답: 3번

풀이와 보기별 판정

「[]」 - 오답. 삭제 후 원소가 앞으로 당겨져 일부를 건너뛴다.

「[2,4]」 - 정답. 처음 1을 제거한 뒤 다음 인덱스의 3을 제거하고 종료한다.

「[1,3]」 - 오답. 제거 대상은 원래 순서의 1과 3이므로 남는 값은 2와 4다.

「[4]」 - 오답. 2도 순회 중 건너뛰어 남는다.

W02Q05

Python3 str "가A", UTF-8 bytes, 같은 UTF-8 데이터를 담은 C++ std::string의 길이를 각각 적어라. 코드 포인트 수/바이트 수/사용자에게 보이는 문자 수를 같은 개념으로 봐도 되는지 설명하라.

모범답안

Python str len은 코드 포인트 2개를 센다. UTF-8 encode 결과는 가 3바이트+A 1바이트로 4바이트다. 해당 바이트를 담은 C++ std::string size도 4다. 코드 포인트는 Unicode 식별 단위이고 UTF-8 바이트는 인코딩 저장 단위다. 결합 문자나 여러 코드 포인트로 이루어진 표현 때문에 사용자가 보는 문자 묶음의 수와 코드 포인트 수도 항상 같지 않다. 필요한 길이 단위를 입력/출력 계약에 명시해야 한다.

10점 자가 채점 기준

- 세 표현의 길이 2/4/4: 4점
- 코드 포인트와 바이트 단위 설명: 3점
- 사용자 문자 묶음과 코드 포인트 차이: 2점
- 길이 단위를 계약에 명시: 1점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 051회차 | 02-06 원리 I

객체지향·상속·다형성·추상화 · 과목 51/80회차 · 기록 묶음 11의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

용어보다 호출 결과의 원리를 이해한다

클래스는 데이터와 동작의 구조를 정의하고 객체는 그 구조를 바탕으로 만들어진 실행 중 실체다. 캡슐화는 내부 표현을 바깥 사용과 분리하고 허용된 인터페이스로 불변 조건을 유지하는 것이다. private 필드를 썼다는 사실만으로 잘 설계된 캡슐화가 완성되는 것은 아니다.

1. 상속과 구성

상속은 기존 타입의 관계를 확장하고 구성은 객체가 다른 객체를 포함해 협력하도록 만든다. "자동차는 엔진이다"는 잘못된 is-a 관계이며 자동차가 엔진을 갖는 has-a 구성 관계다. 모든 코드 중복을 상속으로 해결하면 부적절한 대체 관계가 생길 수 있다. 금융 계좌 잔액은 음수가 가능한지 같은 규칙을 클래스가 보장해야 한다.

예제 A: 검사기의 공통 인터페이스 inspect()를 두고 점등 검사·이미지 검사가 구현하게 하는 것은 기존 장비 경력과 연결된다. 하지만 특정 고객사 연결 방식·파일 포맷을 깊은 상속 층으로 만드는 것이 항상 최선은 아니다. 변경 이유가 다른 기능을 구성하여 교체할 수도 있다. 객체지향의 목표는 "상속을 최대한 많이 사용"이 아니다.

2. 오버로딩과 오버라이딩

오버로딩은 같은 이름에 다른 매개변수 형태를 정의하는 것, 오버라이딩은 상위 타입의 동작을 하위 타입에서 재정의하는 것이다. 반환 타입만 다른 함수를 모두 오버로딩할 수 있다고 단정하지 않는다. 언어의 허용 규칙을 확인한다.

동적 디스패치는 실제 객체의 타입에 따라 적절한 재정의 동작을 선택한다. 정적 타입은 소스에서 변수에 부여된 타입, 동적 타입은 실행 중 객체의 타입이라고 학습한다. 모든 멤버가 자동으로 동적 호출되는 것은 아니다. C++은 virtual, C#은 virtual/override 등의 선언이 중요하다. Java의 일반 인스턴스 메서드 규칙과 static/필드 규칙도 구별한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 052회차 | 02-06 원리ㅍ

객체지향·상속·다형성·추상화 · 과목 52/80회차 · 기록 묶음 11의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 B: C++에서 Base의 virtual f가1, Derived의 override f가2를 반환한다고 하자. Derived 객체를 가리키는 Base&로 f를 호출하면2다. virtual이 없는 Base의 같은 이름 함수라면 선언된 타입을 기준으로 선택될 수 있다. C++에서 기반 클래스 포인터로 파생 객체를 삭제하는 다형적 사용에는 virtual 소멸자 같은 수명 설계도 중요하다.

3. 객체 슬라이싱과 다형성

C++에서 Derived 객체를 Base 값 변수에 복사하면 Base 부분만 담는 객체 슬라이싱이 생길 수 있다. Base 포인터·참조로 파생 객체를 다루는 것과 다르다. 단순히 "상속했으니 어느 변수에서든 파생 동작"이라고 답하지 않는다.

C#·Java의 참조 형 변수에 하위 클래스 객체를 대입하는 일은 C++ 기반 값 객체 복사와 동일한 슬라이싱 모델이 아니다. 값 형·참조 형, 실제 객체 생성·변수 대입·메서드 선언을 함께 본다. Python은 이름의 정적 클래스 타입 선언보다 객체가 제공하는 동작을 활용하는 duck typing을 자주 사용한다.

4. 추상 클래스와 인터페이스

추상 클래스는 직접 인스턴스화하지 못하도록 하고 공통 틀과 일부 구현을 제공할 수 있다. 인터페이스는 제공해야 할 동작의 계약을 표현한다. 특정 언어·버전에서 인터페이스가 기본 구현을 허용하는 등의 세부 기능이 달라져 "인터페이스는 언제나 어떤 구현도 못 가진다" 같은 절대 문장은 피한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 053회차 | 02-06 예제·적용

객체지향·상속·다형성·추상화 · 과목 53/80회차 · 기록 묶음 11의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 C: PaymentProcessor가 authorize(amount)를 제공한다는 계약이 있다면 사용자는 카드·계좌 처리 내부를 모르고 그 계약을 사용할 수 있다. 성공·실패·예외·중복 요청 처리의 의미도 계약에 포함되어야 한다. 이름과 매개변수만 같아도 의미가 다르면 올바른 대체가 아니다.

5. static과 인스턴스, 접근 제한

인스턴스 필드는 각 객체의 상태를 나타내고 static/class 멤버는 타입 차원의 공유 상태나 동작에 사용한다. "static은 모든 경우 메모리 한 번만 쓴다" 같은 표현보다 선언한 클래스·로드·생명주기 범위를 생각한다. static 메서드에는 특정 인스턴스를 자동 제공하는 this가 없으므로 인스턴스 상태를 쓰려면 대상이 필요하다.

손풀이 예제 D: 객체 a와 b의 개별 balance를 각각10,20으로 유지하며 class-level created_count가 두 번 생성 후2라면 balance는 객체마다 다르고 count는 공유 개념이다. Python 클래스 속성에 변경 가능한 목록을 두고 인스턴스에서 append하면 공유가 드러날 수 있어 객체별 목록은 생성자에서 만든다.

6. SOLID를 코드 결과와 연결

단일 책임은 클래스가 한 가지 변경 이유에 집중하는 원칙, 개방·폐쇄는 기존 사용의 변경을 줄이며 확장, 리스코프 대체는 하위 타입이 상위 계약을 깨뜨리지 않는 대체, 인터페이스 분리는 사용하지 않는 동작 의존을 줄임, 의존성 역전은 세부 구현보다 추상 계약에 의존하게 하는 것이다. 이 장은 프로그래밍 관점의 시작이며 자세한 설계·시험은 소프트웨어공학 교재와 연결한다.

완료 기준

상속·구성의 예 하나, 오버로딩·오버라이딩 차이, C++ virtual 호출과 값 복사의 차이를 말한다. 실제 장비 코드의 클래스 하나를 골라 무엇이 객체별 상태이고 무엇이 공유 상태인지 표시하면 이론이 경험과 연결된다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 054회차 | 02-06 직접 문제

객체지향·상속·다형성·추상화 · 과목 54/80회차 · 기록 묶음 11의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q16

오버로딩과 오버라이딩의 설명으로 맞는 것은?

1. 부모 메서드 재정의가 언제나 오버로딩이다.

2. 같은 이름·다른 매개변수 정의가 오버로딩이다.

3. 반환 타입만 다르면 모든 언어에서 오버로딩이 가능하다.

4. 모든 함수 호출은 실제 객체 타입에만 따라 결정된다.

내 선택·근거: _____

B02Q17

C++ Base의 virtual f()가1,Derived의 override f()가2를 반환한다. Derived 객체를 가리키는 Base& r에서 r.f()의 값은?

1. 1

2. 0

3. 반드시 컴파일 오류

4. 2

내 선택·근거: _____

B02Q18

자동차와 엔진의 모델링 관계로 가장 적절한 기본 설명은?

1. 자동차는 엔진을 상속해야 한다.

2. 엔진은 모든 자동차 객체의 정적 메서드여야 한다.

3. 자동차가 엔진을 포함하는 구성이 자연스럽다.

4. 상속을 많이 쓰면 항상 캡슐화가 향상된다.

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 055회차 | 02-06 해설·재현

객체지향·상속·다형성·추상화 · 과목 55/80회차 · 기록 묶음 11의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q16

정답: 2번

오버로딩은 매개변수 형태를 달리한 같은 이름의 함수 정의라는 기본 구분이다. ①는 오버라이딩과 혼동, ③은 언어 제한을 무시한 절대 주장, ④는 static·비virtual·오버로딩 등 선택 규칙을 누락했다.

B02Q17

정답: 4번

virtual 동적 디스패치로 실제 Derived 객체의 재정의 f를 호출한다. ①은 비virtual 정적 선택과 혼동했다. ②은 정의에 없는 값이며③는 유효한 참조와 오버라이딩 사례다. 값 객체 슬라이싱 상황과 구별한다.

B02Q18

정답: 3번

has-a 관계이므로 구성이다. ①은 is-a를 잘못 적용, ②은 엔진 상태·객체 의미를 임의로 없앰, ④는 상속 양과 설계 품질을 무조건 연결한 주장이다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 056회차 | 02-06 원리 I

객체지향·상속·다형성·추상화 · 과목 56/80회차 · 기록 묶음 12의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 손풀이: 호출 종류를 먼저 분류하기

정적 타입 Base,실제 객체 Derived라는 조건만 적지 말고 호출 대상이 인스턴스 virtual 메서드인지,static 메서드인지, 필드인지 먼저 표시한다. Java의 일반 인스턴스 오버라이딩과 C++ virtual 호출은 동적 동작을 설명할 수 있지만 필드 숨김까지 같은 규칙을 적용할 수는 없다.

C++에서 Base 값 변수에 Derived를 복사한 경우는 새 Base 객체가 만들어지는 슬라이싱 모델이다. Base&가 기존 Derived를 가리키는 경우와 그림이 다르다. 포인터·참조·값 중 무엇인지 표시하면 "부모 타입 변수에 자식이 있으니 무조건2"라는 단순 암기를 피한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 057회차 | 02-06 원리ㅍ

객체지향·상속·다형성·추상화 · 과목 57/80회차 · 기록 묶음 12의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

클래스 계층 연습에서는 출력 숫자만 쓰지 말고 어떤 함수 정의가 선택되는지를 한 줄로 적는다. 인터페이스/추상 클래스를 사용해도 실제 객체를 만드는 과정과 오류 계약은 별도로 존재한다. 장비의 검사 전략을 한 가지 바꾸는 예로 상속과 구성을 각각 모델링해 보고 변경 영향이 어떤지 비교한다.

강화 응용: 조건을 바꾸어 다시 증명하기

C++ 다형성에서 호출 방식과 소멸 방식은 별도로 설계한다. Base::f가 virtual이면 실제 객체가 Derived일 때 Base*를 통한 f 호출은 재정의한 Derived::f로 연결될 수 있다. virtual이 아닌 동일 이름 함수는 정적 형의 Base::f를 선택한다. 기반 포인터로 파생 객체를 delete할 설계라면 기반 소멸자를 virtual로 두어야 한다. 그렇지 않은 삭제는 정의되지 않은 동작이므로 '기반 소멸자만 실행되어 조금 누수'라는 확정 결과로 축소하지 않는다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 058회차 | 02-06 예제·적용

객체지향·상속·다형성·추상화 · 과목 58/80회차 · 기록 묶음 12의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

상속 관계가 문법상 가능해도 대체 가능성이 필요한 인터페이스 계약을 깨뜨릴 수 있다. 기본 구현이 '음수 입력도 처리한다'고 약속했다면 파생 구현이 '양수만 허용한다'로 전제조건을 강화하면 기존 호출자가 깨진다. 반환 결과의 보장을 약화하는 것도 위험하다. 구현을 재사용하고 싶다는 이유만으로 상속을 택하지 않고 협력 객체를 보유하는 합성도 검토한다.

Python super()는 단순히 코드에 적힌 첫 부모를 호출하는 문법이라고 일반화하지 않는다. 실제 객체의 MRO에 따른 다음 구현을 찾는다. 다중 상속의 협력 호출에서는 같은 이름 메서드들이 일관된 인수와 super 호출 정책을 공유해야 한다. 기초 문제에서는 클래스 관계·메서드 정의·호출 대상을 모두 제시한 뒤 순서를 추적한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 059회차 | 02-06 직접 문제

객체지향·상속·다형성·추상화 · 과목 59/80회차 · 기록 묶음 12의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q40

C++17에서 Base::f가 virtual이고 Derived가 같은 시그니처로 override했다. Base* p는 살아 있는 Derived 객체를 가리킨다. p->f()는 어느 구현을 호출하는가?

1. Derived::f

2. 항상 Base::f

3. 컴파일 오류, 기반 포인터로 파생 객체를 가리킬 수 없다.

4. 포인터 값에 따라 무작위로 선택한다.

내 선택·근거: _____

B02Q41

C++17에서 Base의 소멸자는 virtual이 아니다. Base*가 동적으로 할당한 Derived 객체를 가리키며 이 포인터로 delete한다. 옳은 설명은?

1. 정의되지 않은 동작이며 결과를 특정 소멸 순서로 고정할 수 없다.

2. 반드시 Base만 정상 소멸하고 나머지 동작도 안전하다.

3. Derived의 소멸자가 virtual이어도 항상 완전히 안전하다.

4. delete는 모든 상속 소멸자를 자동으로 virtual 취급한다.

내 선택·근거: _____

B02Q42

기본 인터페이스가 정수 입력 전체를 허용하고 유효 결과를 반환한다고 약속했다. 대체 가능한 파생 구현의 전제조건으로 적절한 것은?

1. 양수만 허용해 기존 호출자에게 추가 조건을 요구한다.

2. 같은 메서드 이름이면 입력/결과 계약이 달라도 된다.

3. 기존 허용 입력을 모두 받아 기존 결과 보장을 유지한다.

4. 실패 시 다른 의미의 결과를 조용히 돌려도 된다.

내 선택·근거: _____

W02Q06

C++에서 virtual 함수 재정의와 virtual 소멸자의 목적을 구분하라. Base*로 Derived를 참조하는 함수 호출/삭제 상황에서 필요한 계약과 위험을 설명하라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 060회차 | 02-06 해설·재현

객체지향·상속·다형성·추상화 · 과목 60/80회차 · 기록 묶음 12의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q40

정답: 1번

풀이와 보기별 판정

「항상 Base::f」 - 오답. virtual 호출은 객체의 동적 형을 반영한다.

「컴파일 오류, 기반 포인터로 파생 객체를 가리킬 수 없다.」 - 오답. 공개 상속 등 정상 접근 계약에서는 기반 포인터로 참조할 수 있다.

「Derived::f」 - 정답. 재정의된 virtual 함수가 동적 형 Derived에 따라 선택된다.

「포인터 값에 따라 무작위로 선택한다.」 - 오답. 언어의 가상 호출 규칙으로 결정된다.

B02Q41

정답: 1번

풀이와 보기별 판정

「반드시 Base만 정상 소멸하고 나머지 동작도 안전하다.」 - 오답. 비가상 기반 소멸자로 파생 객체를 삭제하는 위험을 단순 순서 결과로 고정했다.

「정의되지 않은 동작이며 결과를 특정 소멸 순서로 고정할 수 없다.」 - 정답. 기반 포인터 삭제를 지원하려면 기반 소멸자의 가상 설계가 필요하다.

「Derived의 소멸자가 virtual이어도 항상 완전히 안전하다.」 - 오답. 기반 소멸자 자체가 가상이어야 해당 기반 삭제가 안전하다.

「delete는 모든 상속 소멸자를 자동으로 virtual 취급한다.」 - 오답. delete가 비가상 소멸자를 자동 가상으로 바꾸지 않는다.

B02Q42

정답: 3번

풀이와 보기별 판정

「기존 허용 입력을 모두 받아 기존 결과 보장을 유지한다.」 - 정답. 기존 호출자의 입력/결과 기대를 깨뜨리지 않는 계약이다.

「양수만 허용해 기존 호출자에게 추가 조건을 요구한다.」 - 오답. 전제조건 강화는 기존 음수/0 호출을 깨뜨린다.

「같은 메서드 이름이면 입력/결과 계약이 달라도 된다.」 - 오답. 이름만 같아도 의미와 계약이 달라질 수 있다.

「실패 시 다른 의미의 결과를 조용히 돌려도 된다.」 - 오답. 결과 보장을 임의로 약화하면 기존 사용자에게 대체 가능하지 않다.

W02Q06

C++에서 virtual 함수 재정의와 virtual 소멸자의 목적을 구분하라. Base*로 Derived를 참조하는 함수 호출/삭제 상황에서 필요한 계약과 위험을 설명하라.

모범답안

virtual 함수는 기반 포인터/참조 호출에서 동적 형의 재정의 구현을 선택하기 위한 것이다. Base::f가 virtual이고 Derived::f가 올바르게 override하면 p->f()는 Derived 구현을 호출한다. 기반 포인터로 파생 객체를 delete할 설계는 기반 소멸자를 virtual로 두어 파생·기반의 소멸이 안전하게 연결되도록 한다. 비가상 기반 소멸자로 Derived를 삭제하면 정의되지 않은 동작이다. 함수 f를 virtual로 했다는 것만으로 소멸자가 자동 virtual이 되지는 않는다.

10점 자가 채점 기준

- 함수 동적 디스패치 목적과 Derived 호출: 4점
- 기반 삭제와 virtual 소멸자 필요: 3점
- 비가상 삭제는 정의되지 않은 동작: 2점
- 일반 함수 virtual과 소멸자 virtual 별도: 1점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 061회차 | 02-07 원리 I

오류·예외·자원 관리·테스트의 기초 · 과목 61/80회차 · 기록 묶음 13의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오류 종류를 먼저 분류한다

문법 오류, 타입·링크 오류, 실행 중 오류, 논리 오류를 구별한다. 실행된다는 사실이 요구대로 계산한다는 뜻은 아니다. 예외 처리는 오류를 없애는 장식이 아니라 오류를 알리고 복구·정리의 흐름을 명시하는 구조다.

1. 예외의 흐름

try에서 예외가 발생하면 일치하는 except/catch로 제어가 이동하고 try의 남은 문장을 그대로 계속 실행하지 않는다. finally는 정리 흐름을 나타내며 일반적인 예외 처리·return 상황에서 실행되지만 프로세스 강제 종료 등의 모든 외부 상황까지 보장하는 만능 규칙은 아니다. 예외를 잡은 후 무시할지 다시 던질지는 함수 계약에 맞춰 판단한다.

예제 A:

```
out = []
try:
    out.append("A")
    int("x")
    out.append("B")
except ValueError:
    out.append("C")
finally:
    out.append("D")
print("".join(out))
```

int("x")에서 ValueError가 발생해 B는 실행하지 않는다. C를 추가하고 finally에서D를 추가하므로 ACD를 출력한다. try가 끝난 뒤의 출력도 실행된다. 다른 예외 타입을 잡도록 바꾸면 잡히지 않을 수 있으나 finally 정리 흐름을 별도로 생각한다.

2. 자원 관리와 RAII

파일·소켓·락·DB연결은 사용을 끝내면 해제해야 하는 자원이다. 메모리 GC가 있다고 그 해제 시점까지 자동으로 원하는 대로 정해지는 것은 아니다. Python with는 context manager의 진입·종료를 통해 사용 범위를 표현하고, C# using은 Dispose 기반 정리를, C++ RAII는 소멸자를 통한 범위 기반 정리를 제공한다.

보완 B08 → 125쪽

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 062회차 | 02-07 원리Ⅱ

오류·예외·자원 관리·테스트의 기초 · 과목 62/80회차 · 기록 묶음 13의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 B: 파일을 열고 중간에 숫자 변환 실패가 생겨도 파일 닫기가 필요하다. with open(path, encoding="utf-8") as f: 안에서 읽으면 일반적인 예외 흐름에서도 context 종료 처리로 닫힌다. open 직후 수동 close를 맨 마지막에만 쓰면 중간 예외에서 close에 도달하지 못할 수 있다. 단, 모든 사용자 context manager가 반드시 파일 닫기 의미를 갖는 것은 아니다.

C++에서는 std::vector·std::string·unique_ptr 같은 소유 구조를 사용하여 new/delete 직접 관리 부담을 줄인다. unique_ptr는 단일 소유를 표현하고 shared_ptr는 공유 소유를 표현한다. shared_ptr를 사용했다고 순환 참조 등 모든 수명 문제가 자동으로 사라지지는 않는다. 모든 객체를 shared_ptr로 바꾸는 것은 설계 원칙이 아니다.

3. 예외를 결과 값으로 바꾸는 계약

읽기 실패에서0을 반환하면 정말 값0을 읽은 경우와 구별하지 못할 수 있다. 성공 여부와 값의 쌍, optional, 예외 등 선택지가 있다. 입력이 항상 올바른 공식 시험 문제에서는 과한 외부 입력 복구가 핵심이 아닐 수 있으나 빈 입력·배열 범위 같은 알고리즘 경계는 꼭 확인한다.

금융 IT 맥락에서는 실패를 삼켜서 성공처럼 보이게 하는 코드는 특히 위험하다. 여기서는 면접 준비가 아니라 프로그래밍 이론의 오류 전파를 이해하는 예시로 사용한다. 변환 실패를0으로 바꾸는 정책을 쓴다면 그렇게 해도 되는 요구가 명확해야 한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 063회차 | 02-07 예제·적용

오류·예외·자원 관리·테스트의 기초 · 과목 63/80회차 · 기록 묶음 13의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

4. 테스트: 정상·경계·반례

새 알고리즘에는 손으로 답을 구할 수 있는 작은 입력, 빈/최소 입력, 동률·중복, 큰 합, 답이 없는 경우를 만든다. 테스트를 많이 돌렸다는 사실만으로 일반적 정확성이 입증되지는 않지만 실패를 발견하고 추적할 근거가 된다. 작은 범위의 완전 탐색 정답과 효율적 풀이를 비교하는 방식도 유용하다.

예제 C: 최대 찾기의 요구가 비어 있지 않은 배열이면 [5]→5,[-3,-8]→-3,[2,2]→2를 확인한다. max_value=0으로 초기화하면 음수만 있는 입력에서0을 반환해 실패한다. 첫 원소로 초기화하는 이유를 불변식으로 설명한다. 빈 배열을 허용한다면 반환 정책을 먼저 정해야 한다.

예제 D: 두 수 합에서 [3,3],target6을 만들면 서로 다른 인덱스를 쓰는 조건을 검증한다. [3],target6은 같은 원소를 두 번 쓸 수 없으면 실패해야 한다. 문제의 "서로 다른 원소/인덱스" 문장을 코드에 반영한다.

5. 디버깅의 순서

기대 결과를 먼저 적고 실제와 처음 달라지는 시점을 찾는다. 입력 파싱→초깃값→조건 경계→업데이트 순서→출력 형식 순으로 분리한다. 디버그 출력은 제출 전에 제거하고 C++ sanitizers 같은 보조 도구가 탐지할 수 있는 메모리 오류와 언어의 보장을 구별한다. 본 교재는 실제 시험에서 해당 도구 사용이 가능하다고 가정하지 않는다.

완료 기준

예제 A의 ACD가 나오는 제어 흐름을 설명한다. 오류를 잡고도 정리를 해야 하는 자원을 하나 적는다. 자신의 함수 하나에 최소 입력·음수·동률·큰 합을 적용하고 기대 답을 손으로 계산한다. "예외가 안 났으니 맞다"라는 판단에서 벗어나면 완료한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 064회차 | 02-07 직접 문제

오류·예외·자원 관리·테스트의 기초 · 과목 64/80회차 · 기록 묶음 13의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q19

Python try에서'A'추가,int('x'),'B'추가; except ValueError에서'C'추가;finally에서'D'추가 후 이어 붙인다. 출력은?

1. ACD

2. ABCD

3. AD

4. CD

내 선택·근거: _____

B02Q20

음수도 허용하는 비어 있지 않은 배열의 최대를 구한다. max_value를0으로 초기화하면 실패하는 대표 입력은?

1. [-3,-8]

2. [1,2]

3. [0,2]

4. [3,3]

내 선택·근거: _____

B02Q21

자원 관리에 대한 설명으로 가장 적절한 것은?

1. 예외가 생기면 파일을 닫을 필요가 없다.

2. GC와 별개로 파일 등 자원 사용 범위를 with/using/RAII 등으로 표현할 수 있다.

3. shared_ptr는 모든 순환 수명 문제를 자동으로 없앤다.

4. try가 있으면 어떤 예외도 자동으로 처리된다.

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 065회차 | 02-07 해설·재현

오류·예외·자원 관리·테스트의 기초 · 과목 65/80회차 · 기록 묶음 13의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q19

정답: 1번

변환 실패로 B를 건너뛰고 except C,finally D를 수행한다. ②은 예외 이후 try 본문을 계속 실행한 오류, ③은 일치하는 except를 누락, ④는 예외 이전 A를 누락했다.

B02Q20

정답: 1번

모두 음수이면 어떤 원소도0보다 크지 않아 잘못된0을 반환한다. 실제 답은-3이다. ②·③·④에서는 우연히 맞지만 일반적 정확성을 입증하지 못한다. 첫 원소 초기화의 근거를 설명한다.

B02Q21

정답: 2번

메모리 회수와 자원 해제는 구별되며 사용 범위를 구조화할 수 있다. ①은 중간 실패의 정리 필요를 무시한다. ③은 공유 소유 순환 등의 한계를 무시한다. ④는 일치하는 catch/except나 전파 정책을 누락한다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 066회차 | 02-07 원리 I

오류·예외·자원 관리·테스트의 기초 · 과목 66/80회차 · 기록 묶음 14의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 검증: 실패 입력을 설계하기

정렬 배열의 lower_bound 반환값이 n인 사례를 만든다. [2,4]에서 target9이면 위치2다. 값을 읽기 전에 위치<len(a)인지 확인해야 한다. 배열 접근 오류가 없더라도 다른 target을 잘못 반환할 수 있으므로 a[pos]==target 조건도 필요하다.

정수 합 문제에[2000000000,2000000000]을 넣으면 수학적 합4000000000이다. C++ int 범위를 넘는지를 확인하고 초기 accumulator의 타입을 적는다. Python에서 int가 자동 확장된다는 사실이 입력 파싱과 조건 오류까지 막아 주지는 않는다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 067회차 | 02-07 원리ㅍ

오류·예외·자원 관리·테스트의 기초 · 과목 67/80회차 · 기록 묶음 14의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

테스트 기록은 입력→기대 답→실제 답→처음 다른 상태 순서로 적는다. 통과한 예제 수 대신 "왜 이 입력이 흔한 버그를 드러내는가"를 설명하면 오답 복습이 학습이 된다. 정답·시간·메모리·출력 계약을 별도 점검한다.

강화 응용: 조건을 바꾸어 다시 증명하기

정상 종료·예외 종료 모두에서 자원을 정리해야 한다. C++ 지역 RAII 객체는 스코프를 벗어나며 소멸자로 자원을 정리하고, Python with는 컨텍스트 관리자의 종료 처리를 호출한다. 그러나 Python에서 파일 객체에 대한 참조를 사라지게 하면 모든 구현에서 즉시 닫힌다고 일반화하지 않는다. 명시적 with가 자원 수명 의도를 표현한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 068회차 | 02-07 예제·적용

오류·예외·자원 관리·테스트의 기초 · 과목 68/80회차 · 기록 묶음 14의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

finally에서 새 return이나 예외를 발생시키면 이전의 return/예외를 덮을 수 있다. Python 함수가 try에서 ValueError를 발생시키지만 finally에서 return 7을 하면 호출자는 7을 받고 그 ValueError는 밖으로 전달되지 않는다. 정리 코드가 오류를 숨기지 않도록 finally에는 자원 정리에 집중하고 반환을 피한다. close 자체가 실패할 가능성도 원래 오류와 함께 설계한다.

테스트는 구현의 내부 식을 같은 코드로 복사해 비교하는 것보다 독립 기준을 둔다. 정렬 테스트는 결과 값뿐 아니라 길이·중복 원소 개수 보존·순서 조건을 확인하고, 작은 입력에서 완전 탐색 기준과 비교할 수 있다. 정의되지 않은 동작의 출력을 관찰해 특정 값이 나왔다는 결과는 언어 표준 안전성의 근거로 쓰지 않는다. 잘못된 코드를 실행해도 우연히 원하는 값이 나올 수 있다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 069회차 | 02-07 직접 문제

오류·예외·자원 관리·테스트의 기초 · 과목 69/80회차 · 기록 묶음 14의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q43

Python3 함수가 try에서 raise ValueError("x")를 실행하지만 finally에서 return 7을 실행한다. 호출 결과는?

1. 7을 반환하며 해당 ValueError는 밖으로 전달되지 않는다.

2. 반드시 ValueError가 호출자에게 전달된다.

3. 반드시 SyntaxError다.

4. ValueError와 7을 동시에 튜플로 반환한다.

내 선택·근거: _____

B02Q44

정렬 함수의 검증으로 가장 적절한 독립 성질 조합은?

1. 출력이 오름차순이기만 하면 원소 누락은 허용한다.

2. 정렬 구현의 비교/교환 코드를 복사해 동일 결과면 충분하다.

3. 이미 정렬된 입력 하나만 통과하면 모든 경우가 증명된다.

4. 출력이 오름차순이고 입력과 원소별 개수·길이가 같음을 확인한다.

내 선택·근거: _____

B02Q45

C++의 정의되지 않은 동작을 포함하는 프로그램이 내 컴퓨터에서 기대값을 한 번 출력했다. 올바른 판단은?

1. 관찰 결과만으로 표준상 안전성이나 다른 실행의 결과를 보장할 수 없다.

2. 같은 컴파일러에서는 영원히 그 값이 보장된다.

3. 최적화를 켜면 정의되지 않은 동작은 자동으로 고쳐진다.

4. 테스트 한 번은 언어 계약보다 우선한다.

내 선택·근거: _____

W02Q07

Python에서 try: raise ValueError("x") / finally: return 7인 함수의 결과를 설명하라. 자원 정리 코드에 return을 피해야 하는 이유와 정렬 함수의 독립 검증 성질 세 가지를 제시하라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 070회차 | 02-07 해설·재현

오류·예외·자원 관리·테스트의 기초 · 과목 70/80회차 · 기록 묶음 14의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q43

정답: 1번

풀이와 보기별 판정

「반드시 ValueError가 호출자에게 전달된다.」 - 오답. finally의 return이 진행 중인 예외를 덮는다.

「반드시 SyntaxError다.」 - 오답. 함수 안의 finally return은 문법적으로 가능하지만 피해야 한다.

「ValueError와 7을 동시에 튜플로 반환한다.」 - 오답. 언어는 예외와 반환값을 자동 튜플로 묶지 않는다.

「7을 반환하며 해당 ValueError는 밖으로 전달되지 않는다.」 - 정답. finally의 제어 흐름이 예외 전달을 대체한다.

B02Q44

정답: 4번

풀이와 보기별 판정

「출력이 오름차순이고 입력과 원소별 개수·길이가 같음을 확인한다.」 - 정답. 순서와 다중집합 보존을 함께 확인해 누락/중복 추가 오류를 잡는다.

「출력이 오름차순이기만 하면 원소 누락은 허용한다.」 - 오답. 빈 배열로 바꾸어도 정렬되어 있으므로 보존 성질이 필요하다.

「정렬 구현의 비교/교환 코드를 복사해 동일 결과면 충분하다.」 - 오답. 같은 결함을 공유할 수 있어 독립 검증이 아니다.

「이미 정렬된 입력 하나만 통과하면 모든 경우가 증명된다.」 - 오답. 역순·중복·빈 값·경계 사례 등의 실패를 검출하지 못한다.

B02Q45

정답: 1번

풀이와 보기별 판정

「같은 컴파일러에서는 영원히 그 값이 보장된다.」 - 오답. 컴파일 옵션/맥락이 바뀌어도 같은 결과라는 보장이 없다.

「관찰 결과만으로 표준상 안전성이나 다른 실행의 결과를 보장할 수 없다.」 - 정답. 표준 보장이 없는 연산은 우연한 실행 결과와 별개로 수정해야 한다.

「최적화를 켜면 정의되지 않은 동작은 자동으로 고쳐진다.」 - 오답. 최적화는 잘못된 프로그램을 안전하게 고치는 계약이 아니다.

「테스트 한 번은 언어 계약보다 우선한다.」 - 오답. 시험 사례가 언어의 정의를 바꾸지 않는다.

W02Q07

Python에서 try: raise ValueError("x") / finally: return 7인 함수의 결과를 설명하라. 자원 정리 코드에 return을 피해야 하는 이유와 정렬 함수의 독립 검증 성질 세 가지를 제시하라.

모범답안

함수는 7을 반환하고 진행 중인 ValueError는 밖으로 전달되지 않는다. finally의 return이 예외 전달을 덮기 때문이다. 정리 코드의 새 return/예외는 원래 실패 원인을 숨길 수 있어 주의해야 한다. 정렬 검증은 출력 오름차순, 길이 보존, 원소별 빈도 보존을 독립 확인한다. 작은 입력의 완전 탐색 또는 신뢰 가능한 별도 기준 구현과 비교할 수 있으나 내부 코드를 그대로 복제한 기준은 같은 결함을 공유할 수 있다.

10점 자가 채점 기준

- 7 반환과 예외 덮임: 3점
- 자원 정리에서 제어 흐름 덮기 위험: 2점
- 오름차순/길이/빈도 보존 세 성질: 3점
- 독립 기준과 구현 복제의 한계: 2점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 071회차 | 02-08 원리 I

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 71/80회차 · 기록 묶음 15의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

마지막 장의 목적

코딩테스트 주력 언어는 C++ 또는 Python 중 하나를 먼저 안정화하고, 같은 문제를 다른 언어로 다시 구현해 표현의 차이를 확인한다. 두 언어를 공부한다고 문제 수가 두 배가 되는 것은 아니다. 필기용 C·Java 코드 해석은 별도 짧은 연습으로 유지한다.

1. C 배열·포인터의 안전한 기본

배열 a에서 a[i]는 유효 범위 안에서 *(a+i)와 대응한다. int a[3]={2,4,6}, int* p=a라면 *(p+1)은4다. 포인터 이동의 한 칸은 해당 타입 원소 하나의 크기이며1바이트라는 뜻이 아니다. 배열 끝 바로 다음 포인터는 만들 수 있는 경우가 있지만 역참조하면 유효 원소 접근이 아니다.

손풀이 예제 A: int가4바이트라는 문제 가정에서 배열 시작 주소1000이면 a[2] 주소는1008이다. int 크기가4라는 조건을 문제에서 주지 않았는데 모든 구현에서 반드시 그렇다고 추측하지 않는다. 두 포인터의 차이는 같은 배열 안 유효한 모델에서 원소 수를 나타내며 바이트 수와 다를 수 있다.

C 문자열은 널 문자로 끝나는 char 배열 모델이다. char s[]="ABC"는 저장에 A,B,C,종료 널의4개가 필요하다. strlen은 종료 널 앞 문자 수3이며 sizeof(s)는 이 선언 지점의 배열 크기4다. char* 포인터 변수의 sizeof는 원래 문자열 길이와 다르다. 이 예제는 ASCII 문자열이다.

2. Java의 인스턴스 메서드와 필드

Java에서 일반 인스턴스 메서드 오버라이딩은 실제 객체에 따라 호출된다. static 메서드와 필드 접근을 같은 동적 디스패치 규칙으로 해석하지 않는다. 부모 타입 변수에 자식 객체를 담았을 때 어느 멤버를 호출하는지 종류별로 읽는다.

예제 B: 부모의 인스턴스 메서드 f가1, 자식이 오버라이드한 f가2라고 하자. Parent x=new Child();x.f()는2다. 부모와 자식이 같은 이름의 필드를 각각 선언했다면 x.field 접근은 메서드 오버라이드처럼 동작하지 않는다. 필드 숨김·static 숨김·인스턴스 오버라이딩을 따로 표로 만든다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 072회차 | 02-08 원리ㅍ

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 72/80회차 · 기록 묶음 15의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

Java 배열·객체 변수는 참조를 담으며 매개변수 전달은 값 전달이다. 참조 값의 복사로 같은 객체를 수정할 수 있지만 함수 안 매개변수에 새 객체를 대입한 것만으로 호출자 변수가 새 객체로 바뀌지는 않는다. Python·C# 일반 참조 형 전달 사례와 비교하되 언어의 문법적 참조 전달이라는 다른 개념을 섞지 않는다.

3. 두 언어로 같은 답을 구현하기

문제: n개의 정수를 받아 짝수의 합을 출력한다. 0 <= n <= 2,147,483,647이고 n개의 입력이 모두 제공된다고 가정한다. 각 입력값과 모든 중간 누적합 및 최종 합이 64비트 부호 있는 정수 범위 안이라는 입력 계약을 둔다. 최종 합만 범위 안에 있어도 중간 합이 범위를 벗어나면 C++ signed overflow 문제가 생길 수 있으므로 중간 합까지 확인한다. Python은 범위 때문에64비트로 잘리는 타입은 아니지만 입력 계약과 알고리즘은 같은 것으로 맞춘다.

```
#include <iostream>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    long long sum = 0;
    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        if (x % 2 == 0) sum += x;
    }
    cout << sum << '\n';
}
```

```
import sys
def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:]
    if len(a) != n:
        raise ValueError("expected n values")
    print(sum(x for x in a if x % 2 == 0))
if __name__ == "__main__":
    main()
```

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 073회차 | 02-08 예제·적용

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 73/80회차 · 기록 묶음 15의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

예제 C: n=5,a=[-4,3,2,7,0]이면 -4+2+0=-2다. 음수의 홀짝 판정에서 %2==0은 두 언어 모두 짝수 여부를 표현하지만 음수 나머지 일반 값은 다를 수 있다. C++는 입력을 한 개씩 받아 추가 저장 O(1), Python 예제는 전체 배열을 받아 O(n) 공간이다. 정답이 같아도 메모리 모델이 같지는 않다.

4. 제출 전 점검표

- 요구 출력이 합인지 개수인지 인덱스인지 확인한다.
- 인덱스0/1 기반, 포함/제외 경계, 정수 범위를 적는다.
- 표준 입력·출력 외 프롬프트를 제거한다.
- n=0·n=1·중복·음수·최댓값을 손검산한다.
- 내장 정렬·컨테이너의 복잡도를 전체 비용에 포함한다.
- C++ 코드가 허용한 버전의 문법인지, Python 재귀 깊이가 안전한지 확인한다.
- SQL 코딩은 알고리즘 구현과 별도 문법·실행 순서를 DB 교재에서 연습한다.

5. 필기 코드 해석의 실제 순서

먼저 언어·버전·타입 크기 가정을 찾는다. 변수의 초기값과 객체 공유 관계를 그린다. 조건·반복·함수 호출을 한 단계씩 표에 기록한다. 언어가 결과를 보장하지 않는 코드는 임의 결과 대신 미정/정의되지 않은 동작 여부를 검토한다. 마지막으로 출력 형식까지 확인한다. 예제를 외우는 것보다 이 순서를 습관화한다.

종합 연습·완료 기준

C 배열 {2,4,6}에서 포인터 p=a+1의 *p는4다. Python의 중첩 얕은 복사에서 내부 객체를 수정하는 문제를 다시 그린다. C++·Python으로 짝수 합 코드를 가린 채 작성하고 [-4,3,2,7,0]→-2, 빈 배열→0을 확인한다. 필기는 결과뿐 아니라 왜 그 값인지 설명하고, 코딩은 새 입력에서도 스스로 구현하면 완료한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 074회차 | 02-08 직접 문제

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 74/80회차 · 기록 묶음 15의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q22

C에서 int a[3]={2,4,6};int* p=a;의 *(p+1)은? 유효한 배열 범위 안이라고 가정.

1. 2

2. 4

3. 6

4. 주소1001

내 선택·근거: _____

B02Q23

C에서 char s[]="ABC"를 선언했다. 이 선언 지점에서 strlen(s),sizeof(s)는? ASCII 문자와 종료 널을 사용하는 모델.

1. 3,3

2. 4,3

3. 3,4

4. 4,4

내 선택·근거: _____

B02Q24

정수[-4,3,2,7,0]에서 짝수 값만 합한 결과는?

1. 2

2. 8

3. 12

4. -2

내 선택·근거: _____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 075회차 | 02-08 해설·재현

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 75/80회차 · 기록 묶음 15의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

먼저 직접 푼 뒤 확인합니다.

B02Q22

정답: 2번

포인터 한 칸은 int 원소 한 개 이동이므로 인덱스1 값4다. ①은 이동을 무시했고③은 두 칸 이동이며④는 역참조 값과 바이트 주소를 혼동했다. int 크기를1바이트로 가정하지 않는다.

B02Q23

정답: 3번

A,B,C 뒤 종료 널을 포함한 배열 크기는4, strlen은 종료 널 앞 문자 수3이다. ①은 널 저장 누락, ②은 두 기준 반대, ④는 strlen에 널을 포함한 오류다. 포인터 매개변수의 sizeof와 이 배열 선언 지점을 구별한다.

B02Q24

정답: 4번

짝수는-4,2,0이고 합-2다. ①는 음수 부호를 빠뜨리거나 음수를 제외, ②은 다른 조건, ③는 모든 정수 전체 합이다. 음수·0의 경계를 함께 검증한다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

프로그래밍 076회차 | 02-08 원리 I

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 76/80회차 · 기록 묶음 16의1/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 앞부분 읽기·용어·조건 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

보강 두 언어 실습: 최대 값과 개수

문제: 비어 있지 않은 정수 목록의 최대 값과 그 값의 등장 횟수를 출력한다. Python에서는 best=max(a),count=a.count(best)로 두 번 순회할 수 있고,C++에서는 한 번 순회하며 갱신할 수도 있다. 두 방식 모두 O(n)이며 순회 횟수 상수는 다르다.

```
def maximum_and_count(a):
    if not a:
        raise ValueError("nonempty input required")
    best = a[0]
    count = 0
    for x in a:
        if x > best:
            best, count = x, 1
        elif x == best:
            count += 1
    return best, count
```

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 077회차 | 02-08 원리ㅍ

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 77/80회차 · 기록 묶음 16의2/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	본문 가운데 부분 읽기·차이·경계 설명
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

[-3,-1,-1,-5]의 답은(-1,2),[4]의 답은(4,1)이다. 첫 원소부터 순회하므로 count 초기0이고 첫 비교에서1이 된다. 첫 원소 뒤에서 순회한다면 count 초기1이 자연스럽다. 두 설계를 섞으면 중복 카운트나 누락이 생긴다. 불변식은 "읽은 구간의 최대와 그 횟수를 저장한다"다. 같은 논리를 C++ long long 타입으로 다시 구현해 언어 문법과 알고리즘 상태를 분리한다.

강화 응용: 조건을 바꾸어 다시 증명하기

C++·C·Java 코드 읽기에서는 대입 앞의 목표 형보다 실제 연산 피연산자의 형을 본다. C++17에서 int a=50000,b=50000; long long p=a*b;는 먼저 int 곱셈을 수행하므로 32비트 int에서 오버플로 위험이 있다. 1LL*a*b나 static_cast<long long>(a)*b처럼 곱셈 전에 확대해야 한다. Java에서도 int 곱셈 뒤 long 대입은 이미 발생한 32비트 순환 결과를 고치지 않으므로 1L*a*b를 쓴다. C++ 부호 오버플로와 Java int 순환은 언어 정의가 다르지만 '연산 전에 확대'라는 구현 지침은 공통이다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 078회차 | 02-08 예제·적용

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 78/80회차 · 기록 묶음 16의3/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	남은 본문·예제의 중간 상태 손추적
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

C의 2차원 행 우선 배열은 원소 크기가 고정이고 선언이 int a[2][3]일 때 다음 행까지 3개 원소 간격이다. 함수 인수에서는 행 길이 등 필요한 형 정보를 계약에 포함한다. Python의 list of list는 각 행 객체 참조를 담는 구조여서 모든 행이 하나의 연속 메모리라는 C 배열 공식을 그대로 적용하지 않는다.

실전 합계 함수는 입력 개수 n, 원소 범위, 모든 누적 중간 합의 범위, 잘못된 토큰 처리, 출력 형식까지 적는다. 최종 합이 안전해도 중간 합이 넘을 수 있다. 비교 정렬의 키에서 a-b로 대소를 판별하면 빨셈 자체가 위험할 수 있어 a<b를 사용한다. 언어별 입력 속도 조정은 정확한 입력 계약과 알고리즘 복잡도를 정한 뒤 적용한다.

오늘 완료 기준

오늘 실제 읽은 절의 용어·조건을 자기 말로 설명하고 해당 예제의 중간 상태를 적습니다. 아직 읽지 못한 절은 다음 시작점으로 남깁니다.

프로그래밍 079회차 | 02-08 직접 문제

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 79/80회차 · 기록 묶음 16의4/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	지정 문제 독립 풀이·서술형 작성
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

오늘 지정 범위: 객관식 3개와 서술형 1개.

정답 페이지를 펼치기 전에 내 선택·근거와 중간 계산을 적습니다.

B02Q46

C++17 int가 32비트이며 a=b=50000이다. 곱을 long long에 안전하게 저장하는 식은? long long은 64비트다.

1. long long p=a*b;

2. long long p=(long long)(a*b);

3. long long p=static_cast<long long>(a)*b;

4. int p=a*b;

내 선택·근거: _____

B02Q47

Java의 int a=50000,b=50000에서 정확한 2,500,000,000을 long p에 저장하는 코드는?

1. long p=a*b;

2. long p=1L*a*b;

3. long p=(long)(a*b);

4. int p=a*b;

내 선택·근거: _____

B02Q48

C++17 long long 누적 합을 쓰는 함수의 안전 계약으로 가장 적절한 것은?

1. 각 입력과 모든 중간 누적 합이 표현 범위 안이어야 한다.

2. 최종 수학적 합만 long long 범위에 있으면 충분하다.

3. 입력 원소만 범위 안이면 합도 항상 안전하다.

4. 오버플로가 나면 나중에 빼기를 하여 복원하면 된다.

내 선택·근거: _____

W02Q08

C++17에서 int a=b=50000, int 32비트·long long 64비트이다. long long p=a*b;와 long long p=static_cast<long long>(a)*b;의 차이를 설명하라. Java에서 같은 문제를 피하는 식과 누적 합 안전 계약도 적어라.

조건·원리·중간 과정·결론 순서로 먼저 답안을 작성합니다.

내 답안: _____

_____

오늘 완료 기준

내 선택과 조건·근거를 먼저 남깁니다. 정답은 바로 다음 별도30분 회차에서 확인합니다. 서술형이 있으면 초안·식·결론을 먼저 적습니다.

프로그래밍 080회차 | 02-08 해설·재현

C·Java 코드 읽기와 C++·Python 실전 구현 연결 · 과목 80/80회차 · 기록 묶음 16의5/5단계

분	할 일
3	앞선 개념 또는 첫 오류 한 가지를 답 없이 회상
25	분리 해설 비교·자가 채점·답 없는 재풀이
2	실제 공부 분·첫 오류·다음 시작점 기록

30분에 멈춥니다. 지정 내용이 남으면 다음 학습일에 같은 회차를 이어갑니다. 아래 자료의 쪽수가 하루에 반드시 끝낼 분량을 뜻하지 않습니다.

오늘 자료

서술형 모범답안과 자가 채점 기준

B02Q46

정답: 3번

풀이와 보기별 판정

「long long p=a*b;」- 오답. int 곱셈이 먼저 수행되어 2,500,000,000은 범위를 넘는다.

「long long p=(long long)(a*b);」- 오답. 오버플로 연산이 먼저이고 이후 형 변환은 되돌리지 못한다.

「long long p=static_cast<long long>(a)*b;」- 정답. 곱셈 전 한 피연산자를 확대해 long long 연산으로 계산한다.

「int p=a*b;」- 오답. 결과가 32비트 int 최댓값보다 크다.

B02Q47

정답: 2번

풀이와 보기별 판정

「long p=1L*a*b;」- 정답. 첫 곱셈부터 long으로 확대해 전체를 long 연산으로 계산한다.

「long p=a*b;」- 오답. 먼저 int 곱셈이 32비트 규칙으로 계산되어 정확한 수를 잃는다.

「long p=(long)(a*b);」- 오답. 캐스트는 int 곱셈 뒤 발생해 이미 틀린 값을 확대한다.

「int p=a*b;」- 오답. int는 해당 양수 값을 표현할 수 없다.

B02Q48

정답: 1번

풀이와 보기별 판정

「최종 수학적 합만 long long 범위에 있으면 충분하다.」 - 오답. 중간에 최댓값을 넘은 뒤 돌아와도 첫 오버플로가 이미 위험하다.

「입력 원소만 범위 안이면 합도 항상 안전하다.」 - 오답. 유효 원소 여러 개의 합은 범위를 넘을 수 있다.

「오버플로가 나면 나중에 빼기를 하여 복원하면 된다.」 - 오답. 정의되지 않은 동작을 후속 연산으로 복구할 수 없다.

「각 입력과 모든 중간 누적 합이 표현 범위 안이어야 한다.」 - 정답. 실제 수행되는 각 덧셈 결과까지 범위를 보장해야 한다.

W02Q08

C++17에서 int a=b=50000, int 32비트·long long 64비트이다. long long p=a*b;와 long long p=static_cast<long long>(a)*b;의 차이를 설명하라. Java에서 같은 문제를 피하는 식과 누적 합 안전 계약도 적어라.

모범답안

첫 식은 a*b를 int에서 먼저 계산하며 2,500,000,000은 32비트 signed int 범위 밖이므로 정의되지 않은 동작이다. 둘째 식은 곱셈 전에 a를 long long으로 바꾸어 b도 확대되고 정확한 2,500,000,000을 계산한다. Java는 long p=1L*a*b; 처럼 연산 전 long을 사용한다. int 곱셈 후 long 캐스트는 이미 계산된 값을 고치지 못한다. 누적 합은 모든 입력뿐 아니라 각 중간 합도 선택한 정수 형 범위 안이어야 한다.

10점 자가 채점 기준

- C++ 첫 식의 int 선계산과 UB: 3점
- 사전 확대 식과 정확한 값: 3점
- Java 1L 식과 사후 캐스트 한계: 2점
- 모든 중간 누적 합 안전 계약: 2점

점수는 학습용 자가 채점이며 실제 시험 배점 재현이 아닙니다.

오늘 완료 기준

첫 오류의 조건을 적고 답을 닫아 재현합니다. 서술형은 모범답안과10점 자가 채점 근거도 남깁니다. 남은 부분이 있으면 같은 회차로 이어갑니다.

근거·범위·검증 상태

현재 GI 장별 교재와 과목별 문제·해설·서술형을 바탕으로 구성했습니다. 기관 공고 범위의 기존 확인 기준은2026-10-04, 이번 PDF 구성일은2026-10-11입니다. 다음 지원 시 새 공식 안내를 다시 확인합니다.

- Python 3.12 공식 문서: 표현식
- Python 공식 문서: 클래스
- Python 공식 문서: 오류와 예외
- Python 공식 문서: copy
- C++ 표준 초안: 정수 타입
- C++ 표준 초안: 함수 호출
- Microsoft C#: 값 형
- Oracle Java Language Specification: 클래스

핵심 보완 찾아보기

원래 회차와 문제 순서는 그대로 따라갑니다. 아래 보완 자료는 해당 개념이 헷갈리는 날의 학습 25분 안에서 본문과 바꾸어 읽습니다. 추가 과제로 더하지 않습니다.

노란색은 정의·공식·판단 조건입니다. 「헷갈리는 지점」은 혼동하기 쉬운 경계를 설명합니다. 강조 개수는 출제 빈도나 배점을 뜻하지 않습니다.

B01. 계산 이전에 범위를 정하는 세 단계

002~008회차 · 자료형과 정수 연산 | 본문 8쪽 → 보완 자료의 같은 번호

B02. 거짓 값과 값 없음은 다른 조건이다

011~013회차 · 조건과 단락 평가 | 본문 20쪽 → 보완 자료의 같은 번호

B03. 스코프와 클로저를 시간축으로 추적하기

027~028회차 · 기본 인수와 스코프 | 본문 43쪽 → 보완 자료의 같은 번호

B04. 얕은 복사를 두 종류의 변경으로 검증하기

032~038회차 · 복사와 메모리 | 본문 50쪽 → 보완 자료의 같은 번호

B05. 실수 오차와 반올림 정책을 분리하기

003회차 · 실수 표현 | 본문 9쪽 → 보완 자료의 같은 번호

B06. vector 삭제 후에는 반환 반복자로 이어가기

038~043회차 · 수명과 컨테이너 | 본문 57쪽 → 보완 자료의 같은 번호

B07. 호출 결과는 virtual·값 복사·const를 함께 읽기

052~058회차 · 다형성과 슬라이싱 | 본문 76쪽 → 보완 자료의 같은 번호

B08. try-else-finally의 실행 경로 표 만들기

061~068회차 · 예외와 자원 관리 | 본문 88쪽 → 보완 자료의 같은 번호

정답·해설은 보완 문제 뒤의 별도 절에서 확인합니다. PDF 책갈피의 「핵심 보완」에서 각 주제로 바로 이동할 수 있습니다.

B01. 계산 이전에 범위를 정하는 세 단계

연결: 002~008회차 · 자료형과 정수 연산 | 본문 8쪽

입력 한 개의 범위, 중간 연산의 범위, 최종 저장 범위를 각각 확인한다.

원리와 판단 조건

n<=200000, |a[i]|<=10^9이면 모든 중간 누적합의 절댓값은 최대 2×10^14다. int가 32비트인 C++ 환경에서는 합계에 long long이 필요하다.

두 int의 곱을 long long에 대입해도 곱셈은 먼저 int에서 일어난다. 한 피연산자를 계산 전에 확대해야 한다.
Python int는 이 범위의 값을 표현하지만 입력 검증·비트 길이 비용까지 없애 주지는 않는다.

적용·손풀이

C++17 (int 32비트 가정)
int a=50000, b=50000;
long long good=1LL*a*b;
// good: 2500000000
// long long bad=a*b; 는 int 곱셈에서 UB 위험
Python: 50000*50000 == 2500000000

헷갈리는 지점

(long long)(a*b)는 곱셈 뒤 변환이어서 늦다. 누적합의 최종값만 범위 안인지 확인하는 것도 부족하다.

직접 확인

int 32비트에서 a=b=100000일 때 long long x=a*b와 long long y=1LL*a*b 중 정확한 계산을 보장할 식과 결과는?

참고: C++17 draft N4659: Expressions (timsong-cpp.github.io)

B02. 거짓 값과 값 없음은 다른 조건이다

연결: 011~013회차 · 조건과 단락 평가 | 본문 20쪽

기본값 선택에서 0·빈 문자열·빈 목록이 유효한 값인지 먼저 정한다.

원리와 판단 조건

Python x or default는 x가 거짓으로 해석되는 모든 경우에 default를 택한다. None만 값 없음으로 쓰는 계약과 다르다.

and/or는 피연산자 객체 하나를 반환하고 오른쪽을 생략할 수 있다. 범위 확인을 배열 접근보다 앞에 두어야 단락 평가가 보호한다.

적용·손풀이

```
value=0
a=value or 99
b=99 if value is None else value
print(a,b) # 99 0
items=[]
print(bool(items) and items[0]>0) # False
```

헷갈리는 지점

x or default를 C#의 null 병합과 동일하게 옮기면 유효한 0을 잃는다. 비트 &는 이 단락 보호를 제공하지 않는다.

직접 확인

재고 0은 유효하고 None만 미입력이다. value=0을 기본 재고 10으로 덮지 않는 조건식을 작성하라.

참고: Python 3.12: Built-in Types (docs.python.org)

B03. 스코프와 클로저를 시간축으로 추적하기

연결: 027~028회차 · 기본 인수와 스코프 | 본문 43쪽

대입이 바인딩 대상을 정하는 시점과 함수가 바깥 값을 읽는 시점을 나누어 본다.

원리와 판단 조건

함수 본문에 x 대입이 있으면 global/nonlocal 선언이 없는 한 x는 지역 이름이다. 같은 함수에서 대입 전에 그 x를 읽으면 바깥 x를 자동으로 사용하지 않는다.

반복 뒤 실행하는 lambda는 보통 나중의 같은 변수 바인딩을 읽는다. 기본 인수 i=i는 정의 시점의 값을 보관하는 한 방법이다.

적용·손풀이

```
fs=[lambda: i for i in range(3)]
gs=[lambda i=i: i for i in range(3)]
print([f() for f in fs]) # [2,2,2]
print([g() for g in gs]) # [0,1,2]
```

헷갈리는 지점

함수가 만들어질 때 바깥 변수의 값이 언제나 자동 복사된다고 생각하지 않는다. 기본 인수로 mutable 객체를 잡으면 그 객체의 내부 변경은 별도 문제다.

직접 확인

x=10이 바깥에 있다. def f(): print(x); x=20을 호출하면 출력은 10인가? 이유를 설명하라.

참고: Python 3.12: Programming FAQ (docs.python.org)

B04. 얕은 복사를 두 종류의 변경으로 검증하기

연결: 032~038회차 · 복사와 메모리 | 본문 50쪽

내부 객체 변경과 바깥 원소의 재대입은 다른 경로를 바꾼다.

원리와 판단 조건

list.copy()는 새 바깥 목록에 기존 원소 참조를 복사한다. 내부 목록을 append하면 두 경로에 보이지만 b[0]=새목록은 b의 경로만 바꾼다.

독립 행은 [[0]*w for _ in range(h)]로 만든다. [[0]*w]*h는 같은 행을 반복 참조한다. 정수 값의 불변성과 행 목록의 가변성을 나누어 그린다.

적용·손풀이

a=[[1],[2]]; b=a.copy()
b[0].append(9)
b[1]=[7]
print(a) # [[1,9],[2]]
print(b) # [[1,9],[7]]

헷갈리는 지점

얕은 복사는 모든 변경을 공유한다는 설명도 틀리다. 어떤 객체를 수정했는지와 어떤 참조를 교체했는지가 핵심이다.

직접 확인

grid=[[0]*2]*2 뒤 grid[0]=[8,8]을 실행했다. grid[1]은 바뀌는가? grid[0][0]=8과 비교하라.

참고: Python 3.12: copy (docs.python.org)

B05. 실수 오차와 반올림 정책을 분리하기

연결: 003회차 · 실수 표현 | 본문 9쪽

표현 오차, 허용 오차 판정, 업무 반올림은 서로 다른 규칙이다.

원리와 판단 조건

십진 문자열을 정확히 받아 계산하려면 Decimal("2.675")처럼 문자열에서 시작한다. Decimal(2.675)는 이미 근사된 이진 float 값을 십진 객체로 옮긴다.

반올림 자릿수뿐 아니라 HALF_UP·HALF_EVEN 등의 동률 정책도 정한다. Decimal도 유한 정밀도 문맥을 사용하므로 모든 나눗셈이 무한히 정확해지는 것은 아니다.

적용·손풀이

```
from decimal import Decimal, ROUND_HALF_UP
x=Decimal("2.675")
print(x.quantize(Decimal("0.01"),
                 rounding=ROUND_HALF_UP))
# 2.68
print(Decimal("0.1")*3 == Decimal("0.3")) # True
```

헷갈리는 지점

round(float,2)와 특정 십진 반올림 정책을 동일하게 취급하지 않는다. 오차 비교도 임의의 10^-9를 모든 크기의 값에 무조건 적용하지 않는다.

직접 확인

Decimal("2.685")를 소수 둘째 자리까지 나타내도록(quantize 0.01) HALF_UP과 HALF_EVEN으로 반올림한 결과를 각각 구하라.

참고: Python 3.12: decimal (docs.python.org)

B06. vector 삭제 후에는 반환 반복자로 이어가기

연결: 038~043회차 · 수명과 컨테이너 | 본문 57쪽

재할당이 없어도 erase 위치 이후의 반복자·참조는 무효화된다.

원리와 판단 조건

push_back은 새 size가 기존 capacity를 넘을 때 재할당하며 기존 원소 주소를 무효화한다. erase는 재할당과 별개로 삭제 위치 이후가 이동한다.

순회하며 삭제할 때 erase가 돌려준 다음 유효 반복자를 사용한다. 컨테이너를 보존해야 하는지 새 결과로 교체해도 되는지도 정한다.

적용·손풀이

```
C++17
for(auto it=v.begin(); it!=v.end(); ){
    if(*it%2==0) it=v.erase(it);
    else ++it;
}
// v={1,2,4,5} -> {1,5}
// 반복 erase는 최악 O(n^2); 대량 필터는 erase-remove 고려
```

헷갈리는 지점

erase 뒤 무조건 ++it를 추가하면 이미 이동한 다음 원소를 건너뛰거나 무효 반복자를 쓸 수 있다. reserve는 삭제 무효화를 막지 않는다.

직접 확인

v={1,2,3}에서 첫 원소 참조 r=v[0]를 저장한 뒤 v.erase(v.begin())를 했다. r로 새 첫 원소 2를 읽어도 되는가?

참고: C++17 draft N4659: vector modifiers (timsong-cpp.github.io)

B07. 호출 결과는 virtual·값 복사·const를 함께 읽기

연결: 052~058회차 · 다형성과 슬라이싱 | 본문 76쪽

부모 타입이라는 말만으로 호출을 결정하지 않고 실제 객체, 참조/값, 함수 시그니처를 확인한다.

원리와 판단 조건

Base&는 기존 파생 객체를 참조할 수 있지만 Base b=derived는 새 Base 값 객체를 만든다. virtual 함수라도 새 Base 객체 안에 파생 부분이 생기지는 않는다.

const 등 시그니처가 다른 함수는 의도한 override가 아닐 수 있다. override 표기를 사용하면 잘못된 재정의를 컴파일 단계에서 드러낼 수 있다.

적용·손풀이

C++17
struct B { virtual int f() const {return 1;} };
struct D:B { int f() const override {return 2;} };
D d; B& r=d; B b=d;
// r.f()==2, b.f()==1

헷갈리는 지점

int f()로 const를 빼고 override를 붙이면 이 예에서는 컴파일 오류다. 기반 포인터로 파생 객체를 삭제하는 설계는 가상 소멸자 계약까지 확인한다.

직접 확인

위 코드에서 B의 f에서 virtual을 제거하고 D의 override 표기도 제거했다. B& r=d; r.f()는 무엇을 호출하는가?

참고: C++17 draft N4659: Virtual functions (timsong-cpp.github.io)

B08. try-else-finally의 실행 경로 표 만들기

연결: 061~068회차 · 예외와 자원 관리 | 본문 88쪽

try가 정상 완료됐을 때의 else와 정리용 finally를 구분한다.

원리와 판단 조건

except는 일치하는 예외를 처리하고 try의 실패 지점 뒤 문장은 건너뛴다. else는 try에서 예외 없이 끝난 경우 실행된다.

finally는 일반적인 정상·예외·return 흐름에서 정리를 수행한다. finally의 return은 이전 반환값이나 진행 중인 예외를 덮을 수 있으므로 정리 코드에는 제어 흐름 변경을 피한다.

적용·손풀이

```
out=[]
try:
    out.append("A"); int("3")
except ValueError:
    out.append("E")
else:
    out.append("S")
finally:
    out.append("F")
print("".join(out)) # ASF
```

헷갈리는 지점

finally를 예외 발생 때만 실행하는 블록으로 이해하지 않는다. 강제 프로세스 종료까지 모든 정리를 보장하는 규칙도 아니다.

직접 확인

예제 int("3")를 int("x")로 바꿀 때 출력과 건너뛰는 블록은?

참고: Python 3.12: Errors and Exceptions (docs.python.org)

보완 문제 정답·해설

문제를 먼저 풀고 첫 오류를 확인합니다. 풀이를 닫은 뒤 같은 조건을 다시 설명할 수 있으면 다음 회차로 넘어갑니다.

B01. 계산 이전에 범위를 정하는 세 단계

y만 이 곱을 long long에서 계산한다. 정확한 값은 10000000000이다. x는 int 범위를 넘는 곱셈을 먼저 시도하므로 특정 순환 값을 정답으로 보장할 수 없다.

B02. 거짓 값과 값 없음은 다른 조건이다

10 if value is None else value를 쓴다. value or 10은 0도 거짓으로 평가해 10을 반환한다. 둘의 차이는 값의 참거짓과 미입력 계약이다.

B03. 스코프와 클로저를 시간축으로 추적하기

UnboundLocalError가 발생한다. 본문의 x=20 때문에 x는 지역 이름이고 print 시점에는 지역 x에 값이 없다. 바깥 값을 변경할 의도라면 명시적 global 등이 필요하지만 가능한 한 인수·반환값으로 계약을 드러낸다.

B04. 얕은 복사를 두 종류의 변경으로 검증하기

행 참조를 교체하므로 grid[1]은 [0,0] 그대로다. 반면 처음 구조에서 grid[0][0]=8은 공유 내부 행을 수정해 두 행 모두 [8,0]으로 보이게 한다.

B05. 실수 오차와 반올림 정책을 분리하기

HALF_UP은 2.69, HALF_EVEN은 2.68이다. 정확히 가운데인 경우 HALF_EVEN은 마지막 유지 자릿수가 짝수가 되도록 정한다. 단순히 출력 자릿수를 두 개로 맞추는 일과 구분한다.

B06. vector 삭제 후에는 반환 반복자로 이어가기

안 된다. erase 위치인 첫 원소 및 그 이후의 참조가 무효화된다. 작업 뒤 v[0]를 새로 조회해야 한다. 값 1을 독립 int 변수로 복사해 두는 것과 참조를 저장하는 것은 다르다.

B07. 호출 결과는 virtual·값 복사·const를 함께 읽기

B::f를 호출해 1이다. 이 호출은 비가상 함수이며 표현식 r의 정적 타입 B를 따른다. 실제 객체 d가 D라는 조건만으로 모든 멤버가 동적 호출되지는 않는다.

B08. try-else-finally의 실행 경로 표 만들기

AEF다. A 후 변환 실패로 ValueError 처리 E로 이동하고 else의 S는 건너뛴다. finally의 F가 뒤따른다. 이를 파일 처리에 적용할 때는 with처럼 자원 범위를 드러내는 구조가 유용하다.